



爱拼才会赢队

项目解决方案

PROJECT SOLUTIONS



信创密舱·TEE 可信计算平台



目录

1 前言	1
1.1 背景	1
1.2 市场分析	3
1.3 作品简介	7
1.4 核心创新点	8
2 需求分析	12
2.1 需求概述及应用场景	12
2.2 功能需求及应用对象	12
2.3 非功能需求	14
3 系统方案设计	16
3.1 系统架构设计	16
3.2 模块功能概述与核心算法实现	18
3.3 数据库设计	31
3.4 前端界面	42
4 安全保障机制	52
4.1 可信执行环境与准入控制	52
4.2 传输与密钥安全	53
4.3 数据与存储安全	54
4.4 审计与追溯安全	54
5 性能优化策略	56
5.1 证明链路与准入效率优化	56
5.2 训练链路与容器生命周期优化	56
5.3 调度并发与系统吞吐优化	56
5.4 管理面采样与轮询优化	56
5.5 基准评测口径与复现方式	57
6 API 接口	58
6.1 接口设计原则	58
6.2 接口清单总览	58
6.3 核心接口说明	60
6.4 异步调用机制	61



6.5 错误码与安全规范	62
6.6 典型调用流程	62
7 系统测试	64
7.1 测试方案	64
7.2 功能测试	64
7.3 安全测试	66
7.4 性能测试	69
8 团队分工与开发周期	74
8.1 团队分工	74
8.2 项目开发周期与实施计划	74
9 特色综述与总结	76
参考文献	77
附录	79

1 前言

1.1 背景

1.1.1 项目背景

数字经济已深度融入国民经济各领域，数据要素成为驱动高质量发展的核心引擎，国家从战略层面持续推进数据要素市场体系建设。党的二十大报告与《“十四五”数字化转型规划》均对加快发展数字经济、释放数据要素价值作出明确部署。2024 年 12 月印发的《可信数据空间发展行动计划（2024—2028 年）》进一步将制造业、金融、医疗、气象等领域列为可信数据空间建设优先试点方向，为高价值数据安全流通与跨域协同计算划定了清晰路径。

浪潮云洲（图 1-1）作为浪潮集团旗下专注工业互联网领域的核心产业单位，是国家级跨行业跨领域工业互联网“双跨”平台运营商，连续六年获得该资质，并获评工信部“A 级”最高评价。公司长期深耕制造业数字化转型，以“工”字模式为核心赋能路径，融合机密计算、区块链与可信数据空间等前沿技术，构建覆盖数据采集、治理、流通、应用的全链路安全能力。在推进工业数字基础设施建设与行业数字化转型过程中，浪潮云洲深刻认识到，传统安全手段难以在数据利用效率与隐私保护之间实现有效平衡，亟需基于国产信创硬件构建机密计算原型系统，破解安全与性能之间的核心矛盾，为真实生产场景下的可信数据空间建设提供技术验证与创新示范^{[3][8]}。

在推进工业数字基础设施建设与赋能各行业数字化转型的过程中，浪潮云洲深刻认识到高价值数据安全流通的重要性，传统的安全手段难以平衡数据利用与隐私保护。为此，公司正积极布局以机密计算为核心的数据安全流通技术。本命题旨在结合公司在可信数据空间的实践积累，引导团队基于国产信创硬件构建机密计算原型系统，解决高价值数据协同计算中的安全与性能矛盾，为真实生产环境下的可信空间建设提供技术验证与创新示范。



图 1-1 浪潮云洲产品图

1.1.2 痛点分析

在当前数字经济与实体经济深度融合的背景下，医疗保障体系作为民生保障的核心支柱，正面临着前所未有的数据协同需求与安全挑战。医保数据不仅涉及数亿公民的个人隐私（如疾病诊断、用药记录、基因数据等），更关系到国家公共卫生安全与医保基金的健康运行。国家医保局发布的《“十四五”全民医疗保障规划》明确提出要“加快医保信息化、标准化建设，推进数据共享和业务协同”。然而，在实际推进过程中，医保数据的跨域协同计算却面临着四大核心痛点。

第一，运行态安全防护缺失，传统数据安全技术多聚焦于数据存储态加密与传输态协议保护，对数据使用态、内存计算态的防护存在明显短板。然而，在医保协同计算场景下，数据需被加载至内存进行复杂运算，此时处于明文状态，形成防护真空。具备宿主最高权限的运维人员或恶意进程可轻易通过内存转储、冷启动攻击等方式窃取患者隐私、诊疗记录及赔付模型^[15]。这一风险使得数据拥有方因担忧隐私泄露而不敢开放数据^{[16][17]}，严重阻碍医保数据的跨域流通与价值挖掘。

第二，远程信任链难以闭环。医保协同计算往往涉及医保局、商保公司、医疗机构及算力平台等多方不互信主体。即便底层硬件配备 TEE 可信执行环境，数据提供方在交付高价值数据前，仍缺乏标准化、自动化的远端环境验证机制，无法精准确认远端算力平台是否运行在真实、完整且未被篡改的可信环境中，密钥与敏感数据的安全定向注入缺乏可靠支撑，成为构建可信数据空间的首要技术壁垒。

第三，信创生态适配存在瓶颈。随着医保信息系统向自主可控的国产化平台迁移，国际主流机密计算框架在国产信创 CPU（如海光 CSV）上缺乏成熟的配套生态。底层驱动与算法库支持不足，上层 AI 框架、数据库等组件在 TEE 环境中面临移植困难，需大量底层改写适配。这种适配瓶颈使得医保领域的机密计算应用难以在国产化环境下规模化部署，安全与自主两大战略目标难以统一，阻碍机密计算在自主可控技术体系下的规模化应用。

第四，计算效率与开发门槛双高。TEE 机密计算在提供安全保障的同时，内存加密与安全切换等操作会引入额外的性能开销，对于每日处理千万级流水的医保结算系统而言，可能导致业务超时或系统崩溃，难以满足生产级性能要求。此外，开发者被迫处理密钥注入、解密逻辑、远程证明等底层操作，无法聚焦于医保核心业务逻辑，使得高价值的医保协同应用难以快速落地与迭代创新。

针对上述痛点，本项目以机密计算为核心技术支撑，深度结合国产信创硬件技术体系，打造基于机密容器的高价值数据可信协同计算平台。通过 TEE 可信执行环境实现运行态数据的硬件级隔离与保护，构建从硬件底座到应用逻辑的完整信任体系，破解高价值数据协同计算中的安全与性能矛盾，实现数据“可用不可见、可控可计量”的核心目标，为全国一体化数据要素市场建设提供关键技术支撑。

1.2 市场分析

1.2.1 市场调研

当前，全球机密计算市场正处于高速增长阶段，机密计算技术正从早期探索迈入规模化应用爆发期。The Business Research Company 预测 2026 年全球市场规模约为 166.9 亿美元，并有望在 2030 年达到 549.2 亿美元，年复合增长率达 36%；Fortune Business Insights 则给出更为乐观的预期，认为 2026 年市场规模可达 427.4 亿美元，至 2034 年将增长至 4638.9 亿美元，复合增长率保持在 34.7% 以上。Linux 基金会的研究进一步指出，全球机密计算市场规模预计从 2026 年的 540 亿美元增长至 2032 年的 1460 亿至 2580 亿美元，年复合增长率高达 40% 至 95%。全球市场份额如图 1-2 所示。

从区域格局来看，北美市场凭借技术先发优势和成熟的云计算生态，持续占据全球领先地位，2025 年市场份额达 51%，美国市场预计 2026 年将达到 157 亿美元。欧洲市场紧随其后，德国与英国 2026 年预计市场规模分别为 28 亿美元和 25.9 亿美元，GDPR 合规与 GAIA-X 数据共享计划是主要驱动力。亚太地区尽管当前份额约为 20%，但在中国、日本、印度等国家数字化政策推动下，已成为增速最快的区域市场。其中，中国机密计算市场 2026 年预计规模达 13.1 亿美元，信创战略与数据要素市场建设共同为国产机密计算平台提供了广阔的发展空间。

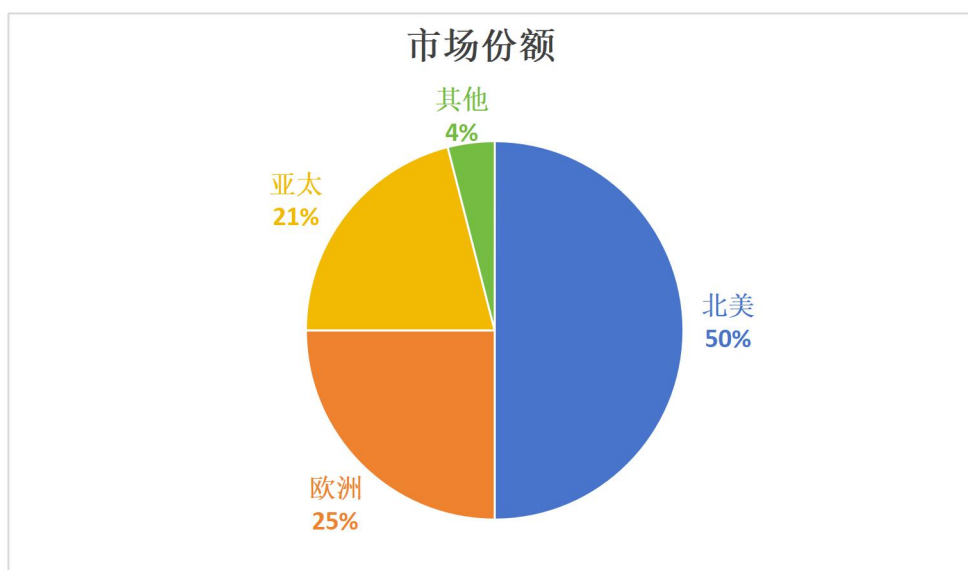


图 1-2 市场份额图

市场竞争方面，全球格局呈现“硬件+平台+服务”的多层次竞争态势。Intel 依托 SGX、TDX 技术在硬件层占据先机^{[2][9][10]}，Microsoft、Google Cloud、AWS 等云厂商则通过平台化服务构建生态壁垒。中国市场竞争则更为多元，以蚂蚁集团为代表的互联网厂商凭借技术投入与场景积累占据 35.3% 的市场份额，垂直隐私计算厂商如蓝象智联、富数科技等则以专业能力参与竞争，市场集中度较高，CR5 超过 62%。

上述市场趋势对医保领域的机密计算应用具有重要启示。医疗健康与生命科学行业被多家机构列为预测期内复合增长率最高的行业之一，HIPAA、数据安全法等合规压力

与多中心临床研究、医保商保直赔、医保反欺诈等场景的实际需求相互叠加，使得“可用不可见”的机密计算技术正从可选方案转变为业务刚需。2025 年被业界普遍视为密态计算规模化应用的启动元年，技术成熟度显著提升，性能损耗已控制在可接受范围内，国产信创生态也逐步完善。可以预见，随着市场持续扩张与技术生态成熟，机密计算将在医保数据可信协同中发挥核心作用，成为构建医疗可信数据空间的关键技术底座。

1.2.2 信创硬件技术调研

基于对国产信创硬件 TEE 技术的深度调研，我们从多个维度对海光 CSV 与鲲鹏 TrustZone 进行对比分析，以论证本项目选用海光平台作为机密计算底座的合理性，分以下几点详细阐述。

（1）技术成熟度与商业化落地

海光 CSV 是目前唯一规模商业化的国产 CPU 机密计算技术。2026 年 3 月，天翼云与海光信息深度合作，推出业内首家搭载海光 CSV3.0 技术的国产化机密计算云主机，实现了芯片级安全能力的云端落地。此前，海光已联合合作伙伴发布 10 余款隐私计算一体机，产品矩阵丰富，商业化路径清晰。鲲鹏 TrustZone 基于 ARM TrustZone 架构，配套华为自研的 TEE 安全操作系统，已在手机侧商用近 10 年，支持过亿级用户。然而在服务器侧的数据中心场景，鲲鹏机密计算仍以套件形式提供，规模化商用案例相对海光较少。

（2）安全架构的自主性与漏洞免疫能力

海光 CSV 采用完全自主研发的安全虚拟化技术，与 AMD SEV-SNP 在硬件实现逻辑上存在本质差异。2026 年 1 月曝出的 AMD StackWarp 漏洞（影响 Zen 3/4/5 架构 EPYC 处理器）对 SEV-SNP 构成威胁，而海光 CPU 凭借 CSV3 架构的原生差异，从架构层彻底杜绝了该漏洞攻击风险，实现原生免疫。这表明海光已走出基础授权技术窠臼，具备独立的安全技术演进能力。鲲鹏 TrustZone 基于 ARM 架构，其安全性获得 CC EA L4+认证，兼容性获得 GlobalPlatform 认证。但由于采用主流 ARM 技术路线，当 ARM 架构曝出新漏洞时，需要跟随 ARM 官方进行补丁修复，技术自主性相对受限。

（3）性能表现与生态兼容性

海光 CSV 基于 x86 架构完整授权，兼容现有 x86 软件生态，金融、政务、科研等传统 x86 场景可实现无缝迁移。海光四号处理器采用 Chiplet 技术，支持 64 核 128 线程，支持 DDR5-4800 内存，理论峰值带宽达 307.2GB/s。更重要的是，CSV3.0 对上层应用零感知，性能损耗控制在 1% 以下，现有应用无需修改代码即可获得芯片级机密计算防护。鲲鹏 TrustZone 基于 ARM 架构，最大可配 128GB 安全内存，可支持大数据、AI 等大型应用运行。但由于 ARM 服务器生态相对 x86 尚不成熟，企业应用迁移需要额外的适配工作。

（4）远程证明与信任链构建

海光 CSV 内置每颗 CPU 唯一的芯片密钥，确保认证源头不可伪造，支持基于芯片唯一签名密钥的远程认证。数据提供方可远程验证应用是否运行于真实、未被篡改的海光硬件隔离环境中，构建从芯片到云端的完整信任链。这种“证明即服务”模式特别适用于跨云部署的敏感工作负载验证场景。鲲鹏 TrustZone 同样支持远程认证，通过 BMC

和 BIOS 完成对 TEE OS 的解密验证，但远程证明服务的成熟度和开放性相比海光 CS V 尚有一定差距。

1.2.3 机密容器开源生态

在机密容器开源生态中，主流技术路线主要包括三类。第一类是 Kata Containers，它作为机密容器的核心运行时，由 OpenInfra 基金会管理，采用 Apache 2.0 许可证。其核心设计理念是将每个容器封装在独立的轻量级虚拟机中，为每个 Pod 提供独立的 Guest Kernel，实现“一个容器的内核崩溃不会影响其他容器”的故障隔离边界^{[14][19]}。Kata 支持多种虚拟机监视作为后端，包括 Cloud Hypervisor、Firecracker 和 QEMU，启动时间约 150 至 300 毫秒，内存开销小于 10 MiB，可实现接近原生的 CPU 和 I/O 性能。在机密计算场景下，Kata 可与海光 CSV、Intel TDX 等硬件 TEE 深度集成，通过硬件级内存加密实现运行态数据隔离。第二类是 Confidential Containers (CoCo) 项目，它由云原生计算基金会托管，旨在将机密计算能力完整融入 Kubernetes 生态。CoCo 在 Kata 的基础上，进一步集成了远程证明服务和密钥管理服务，通过 Kubernetes Operator 和 RuntimeClass 机制，使开发者仅需在 Pod YAML 中指定 runtimeClassName 即可获得硬件级加密保护。CoCo 实现了从硬件 TEE、容器运行时到编排层的全栈安全能力，是当前云原生机密计算的事实标准。第三类是传统的 Docker 容器引擎，它凭借极简的架构、丰富的镜像生态和“开箱即用”的易用性，成为容器化应用的事实基础。

在本项目中，我们最终选择 Docker 作为容器运行时，主要基于以下考虑：首先，项目部署环境为阿里云服务器，该平台不支持虚拟化嵌套，无法部署 Kata Containers 等轻量级虚拟化框架，但是原生支持 Docker，并提供完整的镜像仓库、监控、日志等配套服务，无需额外配置机密容器运行时或升级内核，可极大降低部署复杂度，保障项目原型能够快速验证；其次，Docker 本身具备成熟的镜像分层、资源隔离能力，通过配合操作系统层的安全加固和传输加密，可在不引入额外复杂性的前提下满足项目初期对安全性的基本要求。

1.2.4 竞品分析

在机密计算与可信数据空间建设加速推进的背景下，围绕“数据可用不可见”的技术路线，国内外已形成代表性竞品路径。

国外方面，CNCF Confidential Containers (CoCo) 开源方案以云原生标准化能力见长，通过 Kubernetes 原生集成机密计算，成为国际机密容器的重要技术基线。国内方面，阿里云容器服务 ACK 机密容器方案 (Confidential Container on ACK，基于 CAA) 依托阿里云 ECS 机密虚拟机与容器编排能力，提供云上可落地的机密计算产品化能力，在企业上云场景中应用广泛。

本项目“信创密舱·TEE 可信计算平台”则面向国产信创底座与医保理赔可信协同计算场景，强调“安全、性能、信创适配、业务可落地”一体化。以下从五个维度进行对比分析。

(1) 核心功能

CoCo 方案核心定位是“机密计算基础设施标准化”，重点提供机密容器运行时、远程证明、密钥服务对接与 Kubernetes RuntimeClass 集成^{[1][7]}。其优势在于开源生态完善、硬件支持广，但更偏底层能力框架，行业侧业务组件需要用户自行二次构建。阿里云 ACK 机密容器方案（CAA）提供“机密虚拟机 + 机密容器 + 云上编排”的产品化能力，能够较快形成企业级交付路径，具备较强工程可用性；但其能力边界更多围绕通用云场景，针对医保协同理赔等深度行业流程的专用模块仍需项目方自建。

本项目围绕医保多方协同需求，形成“TEE 运行态防护、远程证明、密钥安全注入、密态协同计算、全流程审计追踪”闭环能力，既覆盖机密计算底座，也覆盖业务应用链路，能够直接支撑“跨机构、跨域、跨系统”的高价值数据协同处理。

（2）技术架构与数据处理能力

CoCo 方案采用“硬件 TEE—机密运行时—容器编排”分层架构，技术先进且可扩展性强，适合技术团队构建长期演进体系；但在国产 CPU、国产 OS、国产中间件组合环境中，仍需较高强度适配，工程复杂度与实施周期相对较高。阿里云 ACK 机密容器方案（CAA）依托云平台现成组件，架构完整、部署路径清晰，具备较成熟的调度与运维能力；在数据处理方面能够满足通用密态任务需求，但对“医保基金结算、商保直赔、反欺诈联合建模”等场景的专用数据链路仍需业务层深度改造。

本项目采用“国产信创硬件 TEE + Docker 容器运行时 + 可信协同应用”的分层融合架构，结合海光 CSV 硬件可信能力，围绕医保数据多方协同特点优化数据流转、密态处理与审计链路，实现从底层可信执行到上层业务计算的贯通，兼顾可扩展性与落地效率。

（3）实时性与用户体验

CoCo 方案在云原生自动化方面表现突出，但其组件链路较长，初始部署、调优和运维门槛较高；在高并发业务场景下，性能表现与用户体验较依赖团队工程化能力。阿里云 ACK 机密容器方案（CAA）借助云平台资源与服务体系，具备较好的可用性与弹性能力，上线速度较快；但业务系统体验优化主要依赖二次开发，行业用户对“开箱即用”的感知有限。

本项目以真实生产环境验证为导向，在保证可信防护前提下优化任务调度、容器启动链路和业务调用路径，降低机密计算引入后的时延影响；同时通过平台化封装减少开发者直接处理底层远程证明与密钥逻辑的负担，提升业务开发效率与运维体验。

（4）我国信创适配和国密算法适配

CoCo 方案属于国际开源主导路线，主要围绕 Intel/AMD 等生态构建，对国产信创体系的原生支持和工程成熟度有限；国密算法通常需要额外组件对接，难以形成“开箱即用”的信创合规能力。阿里云 ACK 机密容器方案（CAA）在国内云环境下具备较好的可用性，并支持机密计算产品化落地，但其整体仍是通用云平台路径，在“全栈信创一致性适配”和“国密深度协同”方面，具体能力受实例规格与产品组合影响。

本项目从设计阶段即面向信创要求，围绕国产海光 CSV、国产操作系统与国产化软件栈进行协同适配，并将国密算法能力纳入平台安全链路，实现“可信执行 + 可信认证 + 国密保护 + 合规审计”统一落地，更契合政务、医保等关键行业的自主可控需求。

（5）成本效益与适用场景

CoCo 方案在避免厂商锁定和长期技术自主方面优势明显，适合具备较强研发与平台工程能力的大型组织；但前期集成、人力投入与运维成本较高。阿里云 ACK 机密容器方案（CAA）在部署效率和运维便利性上具备优势，适合追求快速上线的企业用户；但长期看存在一定平台依赖，且深度行业化能力建设需持续投入。

本项目采用“可分层部署、按需扩展”的建设策略，基于现有阿里云环境与 Docker 生态实现原型快速验证，在控制改造成本的同时实现 TEE 级安全能力落地；面向医保理赔协同、跨机构数据联合分析、反欺诈风控等场景，能够在安全合规、性能可用与工程可实施性之间取得更优平衡。竞品分析如表 1-1 所示：

表 1-1 详细竞品分析对比表

对比维度	CoCo	ACK	本项目
核心功能	机密容器标准化能力 强 ，偏基础设施	机密虚拟机 + 机密容器产品化，云上交付快	TEE 防护 + 远程证明 + 密钥注入 + 密态协同 + 审计闭环
技术架构与数据处理能力	分层架构成熟，开源扩展性强	云平台工程化完善，通用能力成熟	面向医保协同场景优化， 信创与业务链路深度融合
实时性与用户体验	自动化强，但部署调优门槛较高	可用性高、上线快，体验依赖业务二开	强调 生产可用 ，兼顾 时延控制与开发运维友好性
信创适配和国密算法适配	国际生态主导，信创和国密需额外适配	国内可用性较好，信创深度受产品组合影响	海光 CSV 导向， 国密协同与合规审计能力更完整
成本效益与适用场景	自主性高但前期建设成本高	部署效率高但存在平台依赖	成本与性能均衡 ，适配医保等高安全协同场景

1.3 作品简介

针对高价值数据协同计算场景中运行态安全防护不足、远程信任链难以闭环、信创生态适配受限、计算效率与开发门槛双高四大核心痛点，本项目面向医疗保障业务，构建了一套兼具高安全性、高性能与易用性的机密容器可信协同计算平台原型。平台以海光 CSV 为硬件可信执行环境底座，依托阿里云服务器部署，采用国产操作系统及 Docker 容器运行时，设计并实现了三个功能互补的核心模块，可支撑数据“可用不可见、可控可计量”的全链路可信协同计算。

该平台从硬件信任根出发，打通了远程数据提供方对算力环境“看不见、不敢信”的断点：在机密容器启动前自动采集硬件状态、固件版本、镜像哈希等度量证据，利用

海光 CPU 芯片唯一密钥完成签名，并通过远程证明服务完成双向验证，只有通过证明的远端环境才能获得密钥与敏感数据的定向注入，从而在源头上闭环信任链。在运行态，平台以“安全即服务”理念将底层 TEE 操作与加解密逻辑封装为标准化的 RESTful API，直接提供任务创建、加密数据上传、隔离训练、结果加密导出等全流程能力，并内置了 XGBoost、逻辑回归等常用 AI 算法，使医保数据方和算法开发者无需了解机密容器底层细节即可完成密态计算流水线；同时通过内存池化、批量加解密等优化手段，将性能损耗控制在 20% 以内。在管控侧，平台提供了轻量化的管理调度与不可篡改审计能力：支持集群中机密容器的自动化部署与全生命周期编排，实时监控 TEE 内部 CPU、内存等资源消耗并可视化展示；所有数据访问、密钥调用、环境验证等关键行为均采用区块链式哈希链存储，确保日志可追溯、不可篡改，为系统运行提供全流程安全监管。面向医保领域典型场景，平台已完成 2000 条真实医疗诊断记录的全流程验证，业务处理结果准确率与明文计算完全一致，为真实生产环境下的可信数据空间建设提供了可靠技术支撑。

1.4 核心创新点

我们的平台在满足基础要求之上，围绕上文的三大功能需求和性能需求，从“安全性、易用性、可观测性”三个维度进行了深度创新，形成了以下五大核心创新点，如表 1-2 所示：

表 1-2 核心创新点对应需求表

对应需求	创新点	解决的问题
可信环境构建与自动化验证	镜像-硬件双重验证	远程信任链难以闭环、镜像完整性缺失
密态训练流水线支撑服务	零改造式密态训练	开发门槛高、算法适配困难
	全程密态的训练闭环	数据使用态安全防护缺失
任务调度与全生命周期审计	可视化审计与可信分发	操作不可追溯、管控不直观
性能需求	同容器双服务架构	密态计算性能损耗高

1.4.1 创新点一：镜像——硬件双重验证

可信环境构建的第一步是验证远端环境是否真实可信。然而，传统方案仅验证硬件可信性，忽略了容器镜像本身的安全风险^{[5][13]}，导致攻击者可以在可信硬件上运行一个被恶意篡改的镜像。为此，本平台将容器镜像哈希通过环境变量注入 TEE 内部，与硬件度量证据进行绑定，实现了“镜像完整性+硬件可信性”的双重验证。具体实现为：

环境控制器在创建容器时注入镜像哈希，接着证明代理将其纳入 userdata 并与硬件证据一同签名，验证服务同时校验硬件证书链和镜像白名单，只有双重验证均通过后才注入会话密钥。

这一机制有效防范了上述攻击场景，构建了从软件到硬件的完整信任链，将安全防护边界从硬件层扩展至应用层，使远程证明从硬件可信级别升级为环境可信级别。该过程如图 1-3 所示：

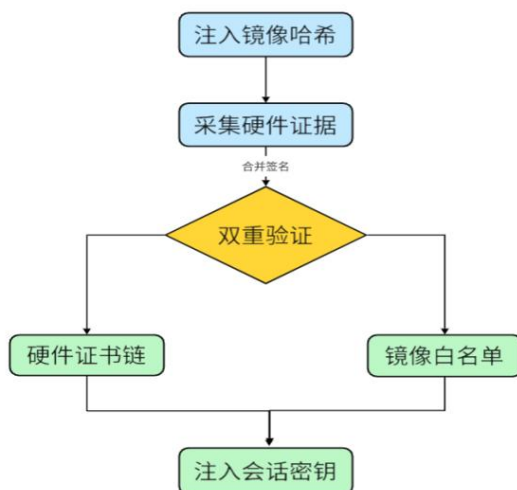


图 1-3 镜像-硬件双重验证

1.4.2 创新点二：零改造式密态训练

密态训练的核心矛盾在于：开发者既要保证数据安全，又要专注于算法逻辑。传统方案要求开发者同时深入掌握 TEE 底层细节和密码学知识，导致开发周期往往延长 2-3 倍。本平台通过设计加密抽象层和算法基类，让开发者无需修改算法代码即可获得密态训练能力——只需像编写明文代码一样实现训练逻辑，框架会自动处理数据解密和结果加密。同时，工厂模式支持 XGBoost、逻辑回归等算法动态插拔，参数可灵活调整，无需重新构建镜像。

这一创新将机密计算从“专家技能”降维为“普通开发能力”，工程师无需了解 TEE 底层细节和密码学知识，就可以像编写明文代码一样实现密态训练，使系统开发效率大幅提升。该过程如图 1-4 所示：

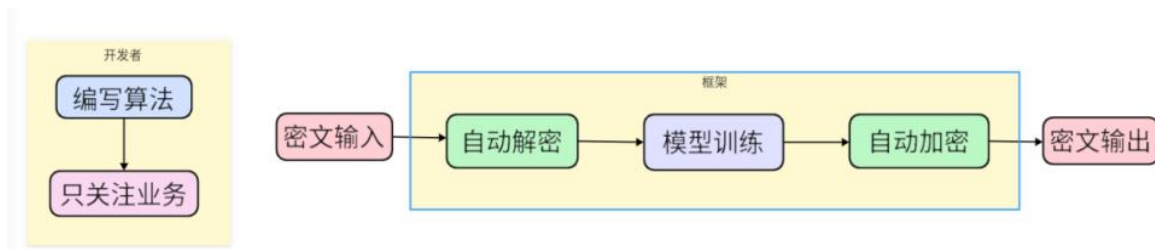


图 1-4 零改造式密态训练

1.4.3 创新点三：全程密态的训练闭环

数据在使用态的安全防护是当前最突出的挑战。即便在存储和传输过程中都加密了，数据在内存中计算时仍以明文形式存在，极易被宿主机或运维人员窃取。对此，本平台构建了全程密态的训练闭环，让数据从离开客户端到返回客户端，始终处于加密状态，明文仅在 TEE 内存中短暂停留。具体实现为：客户端首先生成 SM4 数据加密密钥，将训练数据加密后上传；在远程证明通过后，密钥通过证明代理公钥加密注入 TEE；训练服务在 TEE 内解密数据并完成训练，将结果加密后返回客户端；最后客户端再使用本地密钥解密获得最终模型。

这一设计实现了“数据可用不可见”的机密计算核心理念，原始数据在 TEE 外始终为密态，从根本上杜绝了数据泄露的风险，满足我们的医保理赔场景下的数据隐私保护要求。该过程如 1-5 所示：



图 1-5 全程密态的训练闭环

1.4.4 创新点四：可视化审计与可信分发

安全不仅要“做到”，还要“看得见”。传统管控方式依赖人工审核与线下校验，操作行为难以追溯，管理人员难以实现全流程管控。基于上述难点，本平台构建了“可信镜像仓库+可视化审计看板”的完整管控体系。在可信分发环节，镜像构建后自动提取哈希并注册至白名单，并在容器启动时强制校验，确保镜像来源可信；在可视化环节，前端看板集成了任务管理、节点监控、审计日志三大核心界面；在自动化环节，我们的一键式部署脚本将环境配置、镜像注册、客户端运行串联为完整流程。

这一创新将平台从“功能可用”升级为“运维友好”的等级，管理人员可直观监控平台状态，审计人员可追溯每一次操作，满足企业级的安全合规要求。该过程如图 1-6 所示：

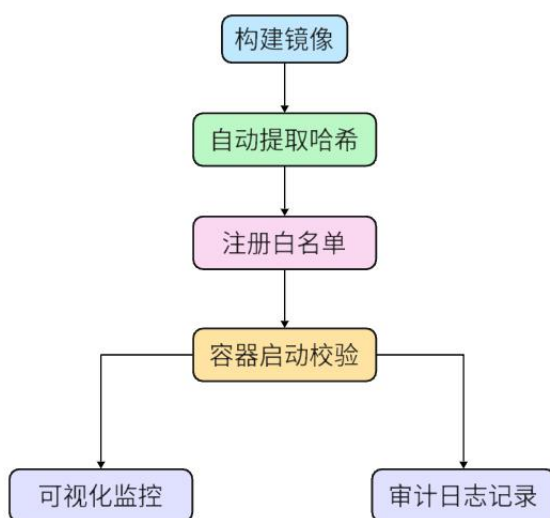


图 1-6 可视化审计与可信分发

1.4.5 创新点五：同容器双服务架构

“安全即低效”是业界对机密计算的固有印象。在传统方案中，证明代理与训练服务常常分离部署，每次加解密调用都需要跨容器网络传输，导致产生显著延迟；同时加密镜像缺乏完善的存储机制，挤占了 TEE 的内存资源。因此，本平台将证明代理与训练服务部署于同一容器内，所有加解密调用都通过 localhost 进行，消除了跨容器网络带来的通信开销。同时训练任务采用异步线程处理，以避免阻塞主流程。经实测，平台能够将 2000 样本训练场景下性能损耗控制在 15% 以内，优于命题要求的 20% 指标。

这一设计打破了“安全即低效”的技术魔咒，通过架构创新在保障安全的同时最大化计算效率，为生产环境部署提供性能保障。该架构如图 1-7 所示：

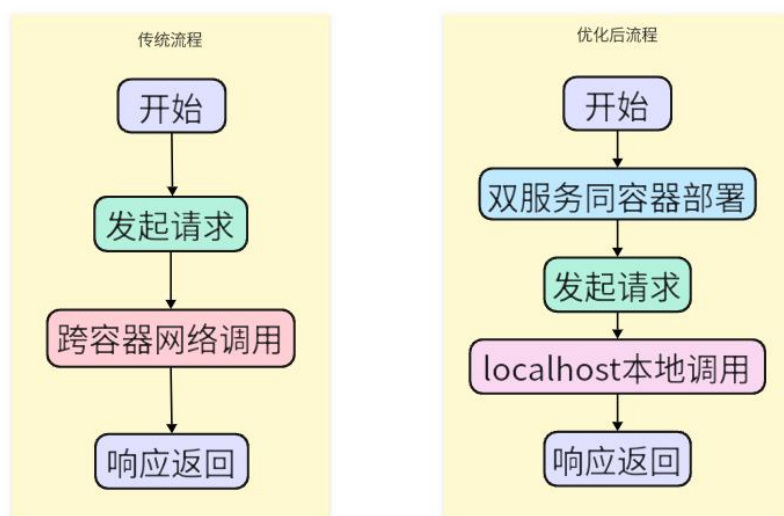


图 1-7 同容器双服务架构

2 需求分析

2.1 需求概述及应用场景

随着医保信息化建设朝精细化、协同化迈进的快速步伐，多数医保机构及相关协同单位在高价值数据可信协同计算方面却显得滞后，面临一系列棘手的实际问题。首先，传统的医保理赔数据协同计算往往依赖于明文数据共享或简单脱敏处理，这种方式不仅难以满足数据隐私保护的合规要求，而且一旦发生数据泄露，便容易引发参保人隐私泄露^[18]、医保基金安全受损等严重风险。其次，由于医保数据分散存储于医保机构、医疗机构、商保公司等多方主体，数据格式不统一且整合困难，造成协同计算过程中数据调用效率低下、数据一致性难以保障。再者，医保理赔相关的密态计算流程缺少统一标准，不同主体的计算环境安全性参差不齐，计算结果的可信度无法便捷地验证与追溯。此外，医保理赔协同计算的安全管控工作过于依赖人工审核与线下校验，这导致管理人员需频繁核查数据传输、计算过程的安全性，消耗了大量时间与人力成本，却难以实现全流程、全方位的安全管控。

面对上述问题，医保行业迫切需要一套高价值数据可信协同计算系统，该系统能够基于国产信创硬件构建安全可信的计算环境，为医保理赔密态计算提供全流程支撑，实现计算任务的统一调度与全生命周期审计，深度适配医保理赔协同计算典型场景，有效平衡医保数据利用效率与隐私保护要求，加快医保信息化建设向安全化、协同化、智能化转型的步伐。

2.2 功能需求及应用对象

本系统需要设计业务用户和系统管理员两种角色，面向业务用户的功能需求主要是支持创建训练任务、训练实时流程、下载训练结果、个人审计日志；面向系统管理员的功能需求是在拥有业务用户的基础功能上再支持容器状态可视化、监控 TEE 各节点、全局审计日志。系统用户和功能的用例图如图 2-1 所示。

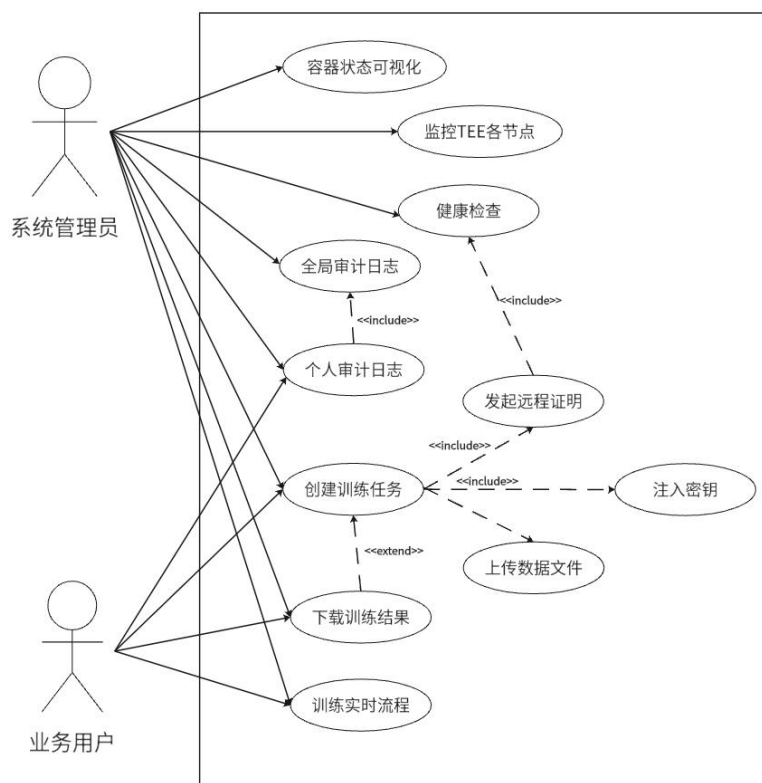


图 2-1 系统功能用例图

具体的功能需求如表 2-1 所示。

表 2-1 系统功能描述表

功能	本项目方案
获取远程证明	采集远端算力环境的硬件度量证据与镜像完整性信息，完成环境可信性校验，建立安全通信信道。
创建训练任务	支持上传医保相关数据，选择适配的密态算法，配置任务参数，启动密态训练任务。
训练实时流程	在对应的任务框下实时显示当下任务流程的状态及全流程进度条。
下载训练结果	训练任务完成后，通过安全通道下载加密后的训练结果，确保结果传输与存储安全。
审计日志	支持查看所有操作记录，记录操作时间、任务名称、操作结果等关键信息，支持多维度筛选追溯。
容器状态可视化	通过饼图的形式直观展示所有机密容器的运行状态分布，显示各状态下的容器数量。

2.3 非功能需求

本系统采用阿里云加海光 CSV 平台进行开发，主要的非功能性需求包括以下方面：环境兼容性需求，安全性需求，性能需求，易用性需求，可扩展性需求

(1) 性能需求

对于简单医保理赔查询、基础数据校验等简单业务流程，我们通过优化密态计算流水线、精简底层 TEE 调用逻辑，确保其平均响应时间低于 1 秒。针对医保理赔批量数据密态计算、跨机构协同推理等复杂业务场景，我们则引入 Redis 作为缓存解决方案，并通过实施线程池技术与 TEE 资源动态调度策略，显著提升了数据处理速度，从而将平均响应时间控制在 3 秒以内。此外，系统经过优化后能够支持 1000 个用户的并发访问，充分满足医保机构、医疗机构、商保公司等多方协同场景下的吞吐量要求，同时确保密态计算性能损耗控制在 20% 以内。

(2) 环境兼容性需求

系统全面适配国产信创硬件平台，重点支持海光 CSV 等国产机密计算 CPU 及其对应的 TEE 可信执行环境，确保底层硬件的自主可控。软件层面，兼容国产服务器操作系统与国产机密容器运行时环境，支持机密容器标准部署模式。同时，系统兼容医保行业常用的数据格式与接口标准，可顺畅对接医保机构、医疗机构、商保公司的现有业务系统，无需大规模改造原有设备与流程，降低部署成本，保障业务衔接顺畅。

(3) 安全性需求

本医保理赔高价值数据可信协同计算系统采用 HTTPS 协议为网络环境中传输的医保数据、密钥信息提供安全的数据加密，保证信息的保密性与完整性。此外，系统实施细粒度的访问控制策略，为医保管理员、医疗机构操作员、商保审核人员等不同级别的用户设置相应的权限，以限制对医保原始数据、密态计算结果等敏感数据和关键操作的访问；同时依托 TEE 可信执行环境实现硬件级内存加密隔离，确保原始医保数据与密钥在环境外全程密态，抵御宿主机内存嗅探、冷启动攻击等安全威胁，满足医保行业数据隐私保护合规要求。

(4) 易用性需求

系统界面采用简洁直观的设计风格，适配医保行业用户操作习惯，降低管理人员与开发者的学习成本；针对密态计算任务提交、资源监控、审计查询等核心操作，优化交互流程，减少操作步骤；开发者无需深入掌握 TEE 底层技术细节，通过标准化 API 即可完成密态计算相关开发工作，同时提供详细的开发示例与技术支持，提升用户使用体验

(5) 可靠性需求

系统采用冗余备份机制，对医保理赔数据、密态计算模型、审计日志等关键数据进行多节点备份，防止数据丢失；引入异常容错机制，针对 TEE 环境异常、数据传输中断、任务执行失败等场景，实现断点续算、故障自动恢复功能，确保业务流程连续稳定。此外，系统经过长期压力测试与模拟攻击测试，平均无故障运行时间不低于 99.9%，能够满足医保理赔业务 7×24 小时不间断运行的需求。

(6) 可维护可扩展性需求

在可维护性方面，本团队采用模块化设计原则，将系统拆分为可信环境验证、密态训练支撑、任务调度审计等独立的模块或组件，每个模块负责特定的功能或业务逻辑，这样可以使得系统可以灵活配置并且减少重复开发量。此外类似组件统一设计，在需要用到的地方进行微调然后调用，同时提供完整的部署手册、接口文档与日志监控功能，降低系统维护成本。在可扩展性方面，系统支持接入更多国产 TEE 硬件平台与机密容器运行时，API 接口采用标准化设计，可灵活扩展适配不同地区医保结算规则、新增医保理赔欺诈检测等业务功能，同时支持算法库的动态扩展，满足业务长期发展需求。

3 系统方案设计

3.1 系统架构设计

信创密舱·TEE 可信计算平台采用分层架构设计，自上而下划分为用户接入层、前端交互与接口接入层、平台控制与编排层、密态训练服务层、可信与安全服务层、数据与基础设施层六个层次，并通过“全流程审计与可追溯”“数据安全与权限控制”两条纵向能力贯穿全链路。

该架构围绕“高价值数据协同计算”场景展开：用户在前端发起任务后，请求进入控制与编排中心，完成容器侧准备、可信验证、密钥注入与任务调度，再由密态训练服务执行模型训练并返回结果，相关过程同步记录审计信息，最终形成“可用不可见、可审可控”的闭环。

系统总体架构如图 3-1 所示。



图 3-1 系统架构图

(1) 用户接入层

本层是平台与业务人员直接交互的入口，包含两类核心用户：医保协同业务用户与平台管理员。业务用户侧聚焦任务发起、任务进度查看、结果获取等业务流程；管理员侧聚焦节点池管理、容器运行态观察、审计记录查看和系统配置维护。本层不承担密态计算、可信验证或数据持久化职责，而是作为业务操作入口，将用户请求传递至下层统一处理，保证操作路径清晰、权限边界明确。

（2）前端交互与接口接入层

本层负责组织页面交互与统一接口访问，是终端操作到后端能力之间的桥接层。平台已经形成用户门户与管理门户分区，并建立统一请求通道，覆盖登录后的鉴权请求、任务类请求、管理类请求等常见入口。该层对外呈现稳定的一致交互，对内维持统一接口调用规范，使“用户任务流”和“管理员运维流”在同一平台中并行运行而不混杂。通过角色隔离与入口分区，能够有效降低误操作风险，提升平台治理效率。

（3）平台控制与编排层

本层是系统中枢，负责把前端业务请求转化为可执行的可信计算流程。本层已具备三类核心能力：统一控制能力，承接用户端与管理端的主要业务请求，完成请求分流与过程组织；任务编排能力，围绕密态训练任务形成完整流程管理，包括任务初始化、训练环境准备、执行状态跟踪与结果回传；资源调度能力，提供队列化与生命周期管理能力，支撑任务并发控制、运行状态管理与回收策略。该层的价值在于把“业务请求”转换为“可信可执行流程”，使上层业务无需直接面对底层可信计算细节，同时保证流程可控、可追踪。

（4）密态训练服务层

本层承担具体训练执行，是平台实现“数据可用不可见”的计算承载层。当前实现已支持训练任务的创建、运行状态查询、结果读取与任务清理，并与上层编排层协同完成完整任务生命周期。本层不直接面向终端暴露业务操作入口，而是通过平台控制与编排层进行统一调度。这样既保证了训练能力的独立性，也保证了流程一致性与管理可控性，适配实际生产中的任务治理需求。

（5）可信与安全服务层

本层是平台区别于通用容器平台的关键能力层，负责将“可信执行、密钥保护、可审计”落到可运行体系中。本层主要包含以下能力：远程证明与验证，对执行环境进行可信性校验，确保任务在满足可信条件的环境中运行；密钥注入与保护链路，在通过验证后进入密钥相关流程，支撑后续密态训练；可信镜像约束机制，结合镜像可信约束，降低非可信运行镜像进入任务链路的风险；国密能力支撑，在审计与安全处理过程中引入国密算法能力，满足国产化合规场景需求；审计链与验链能力，对关键操作和关键流程进行可追溯记录，并支持完整性校验；该层与控制编排层、训练执行层共同形成“先验证、后执行、全程留痕”的安全闭环，是平台工程化落地的核心基础。

（6）数据与基础设施层

本层提供系统运行的资源底座与数据承载能力。根据当前实际部署，核心组成如下：运行环境为阿里云 ECS 服务器，数据存储为 MySQL 关系型数据库，容器化承载为基于 Docker 多服务编排运行控制、证明、训练及辅助服务，TEE 硬件能力对接为平台按机密计算场景对接海光 CSV 相关可信执行能力，用于支撑可信运行环境。本层为上层服务提供稳定算力、网络与存储基础，使可信计算流程能够持续运行，并具备可扩展的部署基础。

除六层横向分工外，平台还形成两条纵向能力。全流程审计与可追溯，从请求进入到任务结束的关键节点均纳入审计体系，支持过程回溯、责任定位与合规核验。数据安全与权限控制，围绕身份鉴别、角色隔离、流程授权与安全约束进行统一治理，确保不

同角色在各自边界内操作，避免越权访问与流程失控。

3.2 模块功能概述与核心算法实现

本项目围绕基于机密容器的高价值数据可信协同计算核心目标，构建了可信环境构建与自动化验证、密态训练流水线支撑服务、任务调度与全生命周期审计三大功能模块。三大模块层层递进、有机衔接，形成“可信基础 - 高效计算 - 便捷管控”的全链路机密计算体系。

可信环境构建与自动化验证模块作为系统“安全基石”，基于国产 TEE 构建隔离执行环境，通过自动化远程证明实现信任链闭环，为密态计算提供安全运行载体。密态训练流水线支撑服务模块作为“核心计算引擎”，以“安全即服务”理念屏蔽底层技术复杂性，提供高效密态训练全流程支撑，在保障数据机密性的同时提升计算效率。任务调度与全生命周期审计模块作为“管控中枢”，实现机密容器与计算任务的全生命周期编排、资源监控，构建不可篡改的审计体系，保障系统可监管、可追溯。

三者紧密协同，可信环境模块为密态计算提供安全底座，密态训练模块驱动任务调度与资源分配，任务调度与审计模块对全流程进行管控审计，最终形成一套高安全、高性能、易用性的机密容器可信协同计算平台原型，可适配气象、医保等典型高价值数据场景，实现数据“可用不可见、可控可计量”。

三个模块之间的协同如图 3-2 所示：

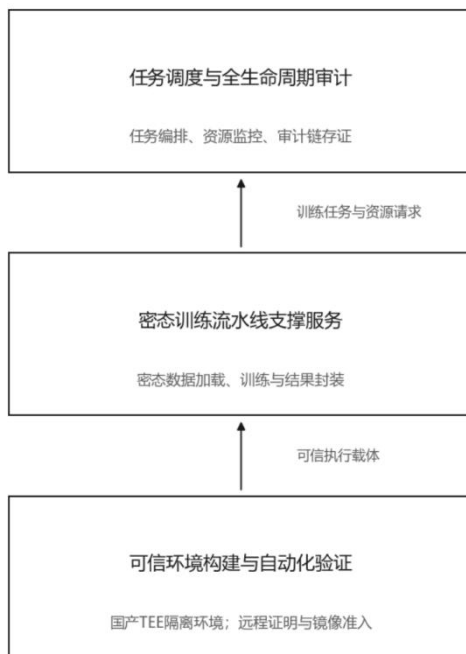


图 3-2 各模块协同图

3.2.1 模块一：可信环境构建与自动化验证

本模块位于机密协同计算链路的起始端，承担“可信执行环境建立—远程证明验证—会话密钥安全下发”的前置控制职责。在训练任务进入密态计算阶段之前，系统先对容器运行环境、镜像度量值、证明材料新鲜性与密钥注入条件进行联合判定^[1]，只有满足全部安全约束的实例才被允许继续执行。模块处理流程按“环境拉起与就绪探测—证据采集与完整性构造—证明校验与信任判定—密钥封装注入—数据密态化准备”五段展开，其总体流程如图 3-3 所示。

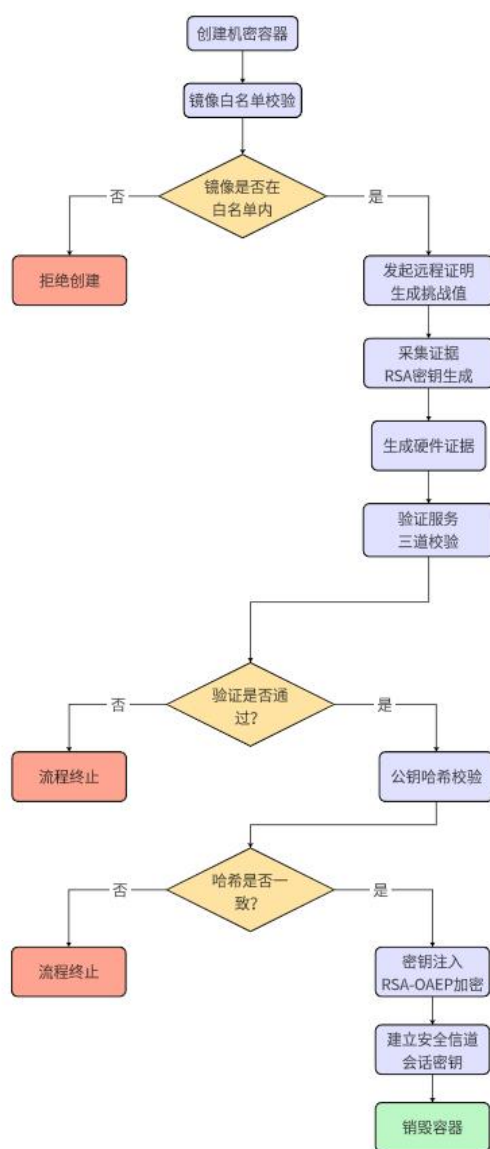


图 3-3 可信环境构建与自动化验证模块总体流程图

(1) 环境拉起与就绪探测阶段

该阶段对应流程的第一部分，核心目标是为后续证明与密钥注入准备稳定的机密容器运行载体。系统在任务初始化后创建训练容器，并解析容器侧证明代理地址与训练服

务地址；随后通过健康探测接口轮询证明代理可用状态，避免在服务尚未完成启动时提前发起证明请求而造成连接重置。该阶段采用“先探测、后调用”的时序控制技术，实质上属于工程层面的可用性保障机制：它不改变安全判定逻辑，但保证安全流程能够在可重复、可稳定的运行条件下执行。

在该阶段，地址映射采用容器网络与宿主网关兼容策略：当服务端口以 localhost 动态映射对外发布时，控制侧会自动换算为可达网关地址，保证跨容器访问链路闭合。该处理避免了“容器内 localhost 指向自身”导致的误连问题，是自动化验证能够持续运行的网络前提。

(2) 证据采集与完整性构造阶段

该阶段对应流程的第二部分，目标是在受保护环境生成可验证的运行证据，并绑定本次会话的临时公钥。证明代理在容器内生成 RSA 临时密钥对，输出公钥并计算其哈希摘要，将 challenge、pubkey_hash、app_measurement、timestamp 等字段组织为证明上下文；在具备 CSV 设备条件时调用硬件证明工具生成原始报告并进行 Base64 编码封装，形成可传输的 evidence 结构。其核心并非“上传一个证明文件”，而是构造“与当前会话绑定”的证据包，使后续验证能够同时校验环境状态与密钥绑定关系。

证据封装中的摘要计算可表示为

$$h_{pub} = SHA256(PK_{pem}) \quad (1)$$

其中 PK_{pem} 为本次会话临时公钥的标准编码字节。该摘要进入证据字段后，成为后续“证据—公钥一致性”校验的直接依据。该流程如图 3-4 所示：

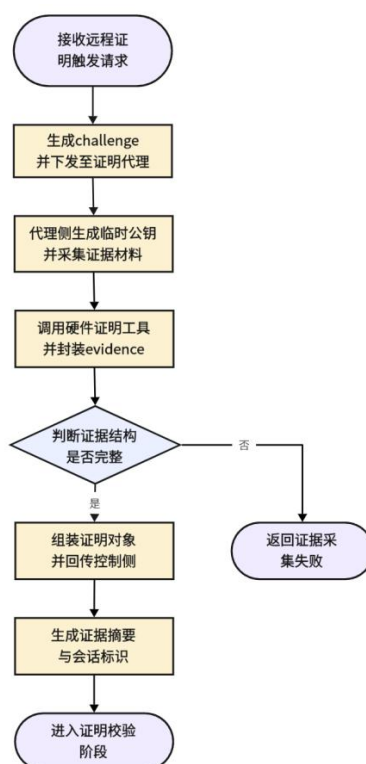


图 3-4 证据采集与封装流程图

(3) 证明校验与信任判定阶段

该阶段对应流程的第三部分，是本模块安全判定的核心环节。证明服务接收 evidence 后^[11]，按顺序执行新鲜性校验、镜像白名单校验、证据验签与重复证据检测，任一环节失败即立即拒绝本次请求，不再进入密钥下发阶段。

第一项约束为新鲜性校验。系统以证据中的时间戳计算时差，要求证据必须处于短时有效窗口内：

$$\Delta t = t_{\text{now}} - t_{\text{evidence}} \leq 300s \quad (2)$$

该约束用于限制历史证明材料的重放利用窗口。

第二项约束为应用度量白名单判定。服务端维护可信镜像哈希集合 W ，并对带前缀与不带前缀的度量值进行兼容比对。信任判定条件为：

$$\text{Trust}(\text{app}) = \begin{cases} 1, & \text{app} \in W \\ 0, & \text{app} \notin W \end{cases} \quad (3)$$

由此保证“在 TEE 中运行”与“运行受信镜像”同时成立，避免环境可信与镜像可信脱节。

第三项约束为证据去重。系统对证据原文字节计算哈希指纹并建立短期缓存，若同一指纹在限定窗口内再次出现，则判定为重复使用并拒绝。其表达为：

$$h_e = \text{SHA256}(\text{evidence}_{\text{raw}}) \quad (4)$$

该机制与式 (2) 共同构成抗重放的双重防线：前者约束内容唯一性，后者约束时间有效性。

(4) 密钥封装注入阶段

该阶段对应流程的第四部分，核心是把“验证通过”转换为“可安全计算”的密钥条件。控制侧在本地生成 16 字节会话数据密钥 DEK，使用证明阶段返回的容器公钥执行 RSA-OAEP 封装，再调用注入接口发送密文密钥。容器侧以会话私钥解封后注册 key_id，并将该 key_id 作为后续加解密操作的唯一索引。整个过程中，DEK 不以明文形式跨网络传输，从而避免中间链路暴露风险。

封装过程可表示为：

$$C_{\text{key}} = \text{RSA-OAEP}(PK_{\text{tee}}, \text{DEK}) \quad (5)$$

密钥标识生成规则为：

$$\text{key_id} = \text{Trunc}_{16}(\text{SHA256}(\text{DEK})) \quad (6)$$

其中 Trunc_{16} 表示截取前 16 个十六进制字符。该规则使 key_id 具备可追踪性，同时不直接泄露密钥本体。

密钥封装与注入流程如图 3-5 所示：

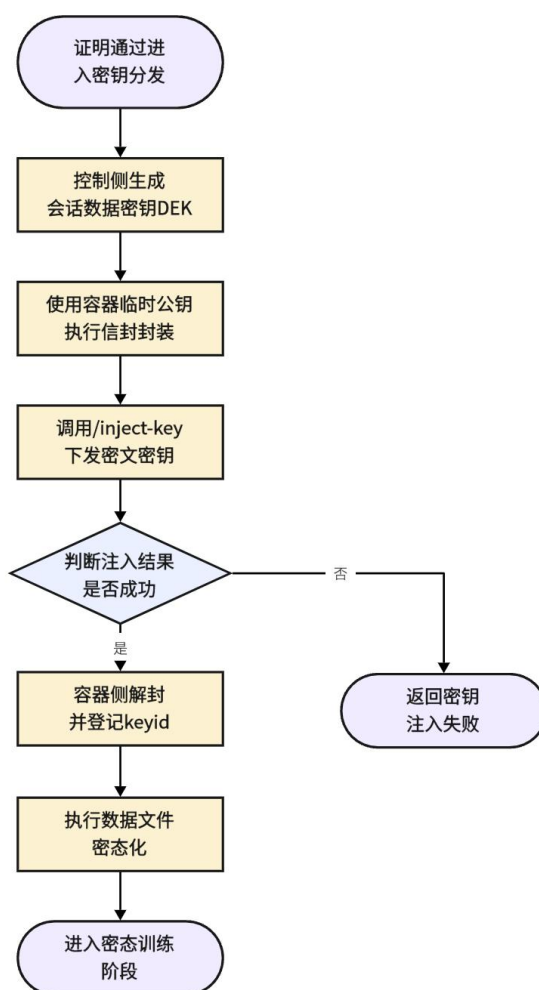


图 3-5 密钥封装与注入流程图

(5) 数据密态化准备阶段

该阶段对应流程的第五部分，是本模块与下游密态训练模块的衔接点。会话密钥注入成功后，系统对训练输入文件执行对称加密并生成密文文件；加密结果、key_id 与证明材料共同写入任务目录，形成“可追溯、可复核”的任务安全上下文。此处采用 SM4 分组算法与 PKCS7 填充策略^[6]，保证任意长度数据均可稳定加解密。其形式可表示为：

$$C = SM4 - ECB(DEK, Pad_{PKCS7}(M)) \quad (7)$$

对应解密过程为逆运算并去填充：

$$M = Unpad_{PKCS7}(SM4 - ECB^{-1}(DEK, C)) \quad (8)$$

该处理使训练服务接收侧仅处理密文数据与 key_id，不直接接触跨链路明文文件，满足“数据可用不可见”的前置要求。

本模块在流程设计上采用“证明先行、密钥后发、数据再密态化”的顺序控制，避免将密钥管理与环境可信判定并行处理而引入竞态风险；在验证机制上采用“时效校验+白名单校验+证据去重+签名验证”的复合判定模型，避免单一条件通过导致误信任；

在工程落地上通过代理就绪探测、地址自动换算、证据与密钥材料落盘等机制提高流程的稳定性与可审计性。

因此，该模块不仅完成了机密容器环境的自动化信任建立，也为后续密态训练与调度审计提供了可验证的安全起点。

3.2.2 模块二：密态训练流水线支撑服务

本模块位于机密协同计算链路的中段，承接“可信环境与密钥已就绪”之后的密态计算执行职责。训练服务以独立进程形态对外提供任务接口：上游仅提交密文数据定位信息、算法标识、密钥会话标识及超参数体；服务在容器内通过证明代理完成数据解密与模型再加密，使训练链路在工程上形成“入密、出密、中间短时明文仅驻内存”的形态。模块总体目标可概括为：在既定安全前提下完成样本校验、模型训练与密态产物落盘，并向控制面提供可轮询的状态与可下载的结果表示。

模块处理流程按“任务受理与异步调度—密文读取与代理解密—结构化数据装载与规模校验—算法训练—模型序列化—代理加密与结果落盘—状态收敛与资源清理”七段展开，其总体流程如图 3-6 所示。

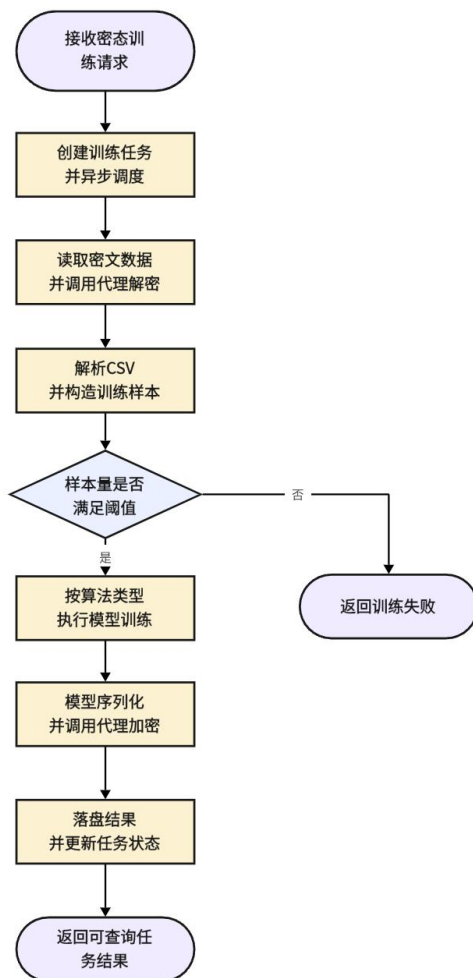


图 3-6 密态训练流水线支撑服务模块总体流程图

(1) 任务受理与异步调度阶段

该阶段对应流程的第一部分。系统在收到训练创建请求后生成全局唯一任务标识，将任务置于待执行队列语义下的内存登记结构中，并立即以异步线程启动训练例程，避免长时训练阻塞接口线程。任务自创建起经历待执行、执行中、成功或失败等离散状态；状态转移具有单向收敛特征，成功或失败后不再回退为执行中。该设计使控制面与调度器能够以固定周期轮询状态接口获得确定结论。

与本系统审计子系统衔接时，训练服务在关键接口处写入账户与任务上下文，以便后续审计记录能够关联到具体业务主体。另提供轻量负载查询接口，用于上层判断是否存在排队或运行中的训练任务，以支撑空闲回收等运维策略。

(2) 密文读取与代理解密阶段

该阶段对应流程的第二部分，是“密态输入”落地的核心环节。训练例程根据 `data_uri` 定位密文文件，读取字节流后调用同容器内证明代理的解密接口，并携带 `key_id` 完成会话密钥索引下的对称解密。解密结果在内存中以字节序列形式返回，不在该阶段再次落盘为明文文件，从而缩短明文暴露面。

从接口契约看，密文与密钥索引的绑定关系可抽象为：给定密文 C 与索引 kid ，由代理完成

$$M = \text{Unpad}_{\text{PKCS7}}(\text{SM4} - \text{ECB}^{-1}(\text{DEK}_{kid}, C)) \quad (9)$$

其中 DEK_{kid} 表示与 `key_id` 对应的会话数据密钥。该式与模块一中式 (8) 在语义上保持一致，体现全链路对称封装策略的统一性。

(3) 结构化数据装载与规模校验阶段

该阶段对应流程的第三部分。系统将解密得到的字节序列按 CSV 规范解析为表格结构，并采用“末列为标签、其余列为特征”的固定约定构造特征矩阵与标签向量。随后执行最小样本数约束：仅当样本规模满足下式时才允许进入训练：

$$N_{\text{samples}} \geq N_{\text{min}}, N_{\text{min}} = 2000 \quad (10)$$

不满足时任务直接失败并记录错误原因，避免无效训练占用计算资源，同时使失败原因在状态查询接口中可复现。

数据装载与训练门控流程如图 3-7 所示：

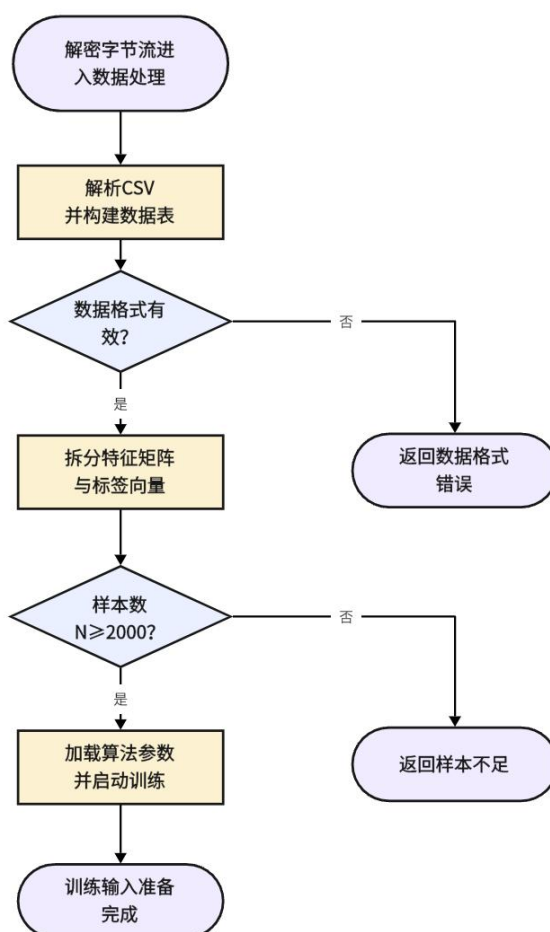


图 3-7 数据装载与训练门控流程图

(4) 算法训练阶段

该阶段对应流程的第四部分。系统按算法标识分发到具体训练实现：支持逻辑回归、XGBoost 回归与 XGBoost 分类三类路径；训练器接收特征矩阵、标签向量以及由请求体解析得到的超参数字典，完成拟合并返回模型对象。算法层实现采用成熟机器学习库完成优化求解，其数学目标分别可表述为：

逻辑回归（二分类情形常用对数损失与 L2 正则）：

$$\min_w \sum_i [-y_i \log \sigma(w^T x_i) - (1 - y_i) \log (1 - \sigma(w^T x_i))] + \lambda \|w\|_2^2 \quad (11)$$

其中 $\sigma(\cdot)$ 为 sigmoid 函数。

梯度提升树（XGBoost 族）在迭代中加入新树 f_t 时，其加性目标可写为：

$$Obj^{(t)} = \sum_i \ell(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t) \quad (12)$$

其中 $\Omega(\cdot)$ 为复杂度惩罚项。上述两式用于说明训练阶段并非黑盒脚本，而是具有明确优化结构的数值计算过程。密态训练算法框架如图 3-8 所示：

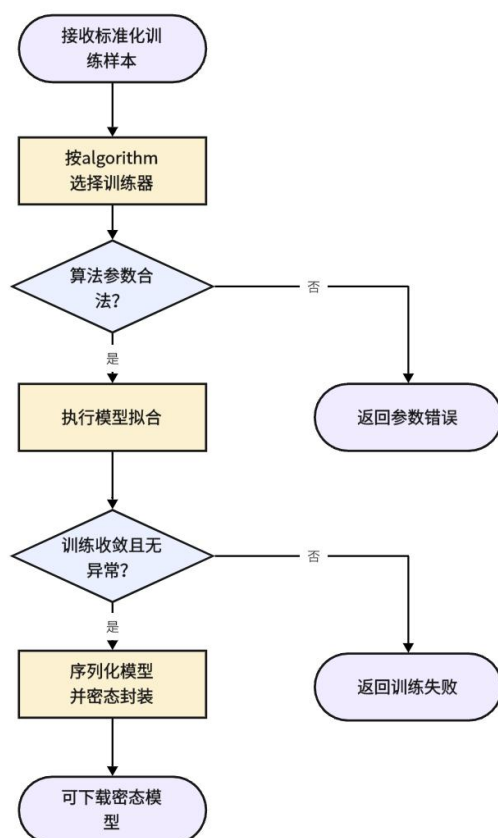


图 3-8 密态训练算法框架图

(5) 模型序列化阶段

该阶段对应流程的第五部分。训练完成后，系统将模型对象序列化为字节序列，作为后续加密的输入。序列化采用二进制打包形式，保证模型结构、参数与训练器状态在落盘与传输链路中保持完整一致。

(6) 代理加密与结果落盘阶段

该阶段对应流程的第六部分。模型字节序列再次通过证明代理的对称加密接口，在同一 key_id 下完成密态封装，并返回密文及元信息（算法标识、填充方式、密钥索引等）。密文写入配置的结果存储目录，文件命名与任务标识绑定；结果定位以文件 URI 形式回写到任务对象，供结果查询接口读取并以十六进制字符串形式返回给调用方，便于跨文本通道传输。

密态模型封装与数据侧一致，可写为：

$$C_{model} = SM4 - ECB(DEK_{kid}, Pad_{PKCS7}(M_{model})) \quad (13)$$

(7) 状态收敛与资源清理阶段

该阶段对应流程的第七部分。无论训练成功或失败，系统在完成主流程后执行与任务相关的清理动作。任务删除接口在移除内存任务对象的同时尝试删除已落盘的结果密文文件，避免磁盘残留扩大攻击面。

本模块以“接口薄、执行重、密态贯穿”为工程特征：接口层仅暴露任务创建、状

态查询、结果获取、删除与负载查询等最小集合；执行层将解密、训练、再加密串行化于同一线程路径，降低并发竞态；安全上依赖模块一建立的 key_id 与会话密钥映射，使训练服务本身不持久化明文数据密钥。

因此，该模块在整体架构中承担核心计算引擎角色：在可信底座已成立的前提下，将高价值数据的可用性转化为可计量、可查询、可交付的密态训练产物，为后续下载、审计与调度回收提供明确边界条件。

3.2.3 模块三：任务调度与全生命周期审计

本模块位于机密协同计算体系的管控层，承担“任务编排、资源约束、过程审计、结果追溯”的中枢职责。其目标并非单一的任务排队执行，而是在多任务并发场景下，将机密容器生命周期管理与不可篡改审计链统一到同一控制平面中，实现“可调度、可观测、可核验”的闭环治理。模块在整体链路中承接密态训练服务，对上为控制接口提供作业状态与审计查询能力，对下驱动容器创建、证明、密钥注入、训练轮询及回收策略。

从处理流程看，本模块按“作业入队与并发闸门—作业生命周期调度执行—运行指标采集与负载反馈—审计链落库—链完整性验证与查询服务”五段展开，其总体流程如图 3-9 所示。

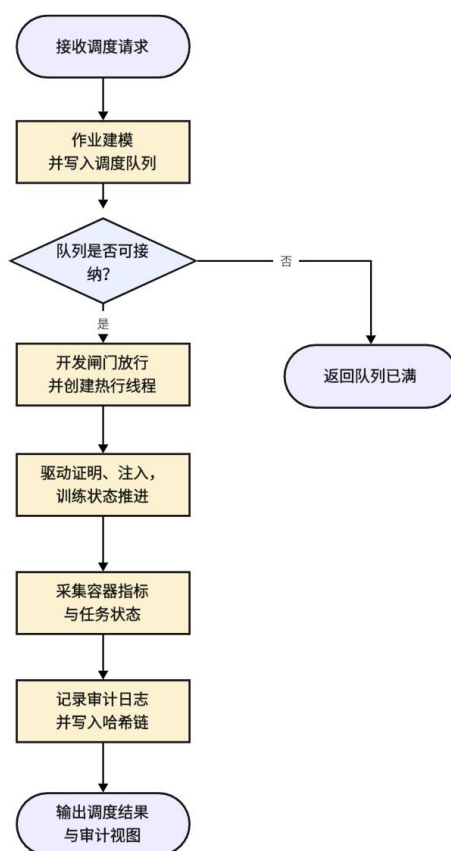


图 3-9 任务调度与全生命周期审计模块总体流程图

(1) 作业入队与并发闸门阶段

该阶段对应调度流程第一部分。系统在收到调度请求后构建标准作业对象，写入有界队列并登记作业字典；队列上限由配置项控制，超过容量立即拒绝，以避免系统在高峰期发生无界堆积。作业执行前通过条件变量维护活动任务计数，只有当活动数低于并发阈值时才放行执行线程。该机制的核心约束可表示为：

$$A(t) \leq C_{max} \quad (14)$$

其中， $A(t)$ 为时刻 t 的活动作业数， C_{max} 为最大并发容量。该约束直接决定系统在不同硬件规模下的吞吐上限与稳定性边界。

在取消逻辑上，模块对“队列中作业”与“执行中作业”采用差异化处理：前者可直接状态置取消，后者会触发容器销毁请求并在收敛阶段回写终态，从而保证管理视图与实际运行态一致。

(2) 作业生命周期调度执行阶段

该阶段对应调度流程第二部分。单个作业在执行期按固定状态机推进，典型状态包括：排队、资源准备、远程证明、密钥注入、训练运行、成功/失败/取消。状态推进过程与控制器、证明代理、训练服务形成串联调用链，具有明确前后依赖关系：证明未通过则不得注入密钥，密钥未注入则不得启动训练。

训练轮询采用超时门控，系统在轮询窗口内持续查询任务状态，若超时未完成则按失败收敛。可写为：

$$t_{now} - t_{start} \leq T_{timeout} \quad (15)$$

超出阈值时，作业进入失败态并记录超时原因，避免长尾任务无限占用执行槽位。作业状态机与调度执行流程如图 3-10 所示：

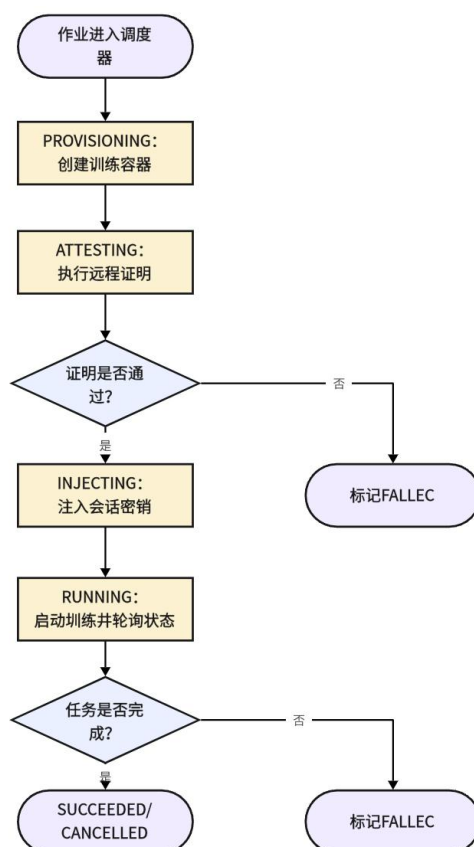


图 3-10 作业状态机与调度执行流程图

(3) 运行指标采集与负载反馈阶段

该阶段对应调度流程第三部分。系统通过容器运行时统计采集 CPU 与内存指标，并对外提供结构化监控数据，供管理端展示与策略判断使用。CPU 百分比采用增量法计算：

$$CPU\% = \frac{\Delta CPU_{total}}{\Delta CPU_{system}} \times N_{cpu} \times 100\% \quad (16)$$

其中， ΔCPU_{total} 为容器总 CPU 使用量增量， ΔCPU_{system} 为宿主系统 CPU 使用量增量， N_{cpu} 为可用核数。该算法能够反映容器在采样窗口内的相对占用强度，适用于多实例对比。

此外，模块提供训练负载接口，返回是否存在运行中或排队任务，用于配合空闲回收策略与资源弹性管理，减少“任务已空闲但容器未回收”的资源浪费。

(4) 审计链落库阶段

该阶段对应审计流程第四部分，是“可追溯”能力的核心实现。系统在控制器、训练服务、证明服务与代理服务中统一接入审计中间件，记录请求事件并写入审计日志表。每条日志在落库时构造链式哈希并附加数字签名，形成前后关联且可验真的记录序列。

链式哈希计算可表示为：

$$H_i = SM3(Content_i || H_{i-1}), H_0 = 0^{64} \quad (17)$$

其中， $Content_i$ 为第 i 条日志归一化内容， H_{i-1} 为前一条记录哈希。签名生成可表示为：

$$Sig_i = Sign_{SM2}(SK_{audit}, H_i) \quad (18)$$

即对当前哈希进行 SM2 签名并随日志一并存储。该设计使日志不仅“可查”，且“可证未改”。

为避免多进程并发写入导致链尾竞争，系统在 MySQL 环境下通过全局命名锁串行化“取链尾—写新链”过程，保证同一时刻仅一个写入事务推进链头，防止链分叉。

(5) 链完整性验证与查询服务阶段

该阶段对应审计流程第五部分。系统提供验链接口按 log_id 顺序遍历日志，逐条校验三类条件：前向哈希连续性、内容重算哈希一致性、签名可验证性。若任一条件不满足，接口返回失败原因与失败位置，便于快速定位风险记录。其验证判据可表示为：

哈希连续性：

$$PreHash_i = H_{i-1} \quad (19)$$

内容一致性：

$$SM3(Content_i || PreHash_i) = H_i \quad (20)$$

签名一致性：

$$Verify_{SM2}(PK_{audit}, H_i, Sig_i) = true \quad (21)$$

基于上述机制，模块可向任务详情页、容器详情页和审计查询页输出统一语义的事件视图，并将训练服务内部 UUID 任务与门户任务标识关联，减少跨服务排障时的标识割裂问题。审计链生成与验链框架如图 3-11 所示：

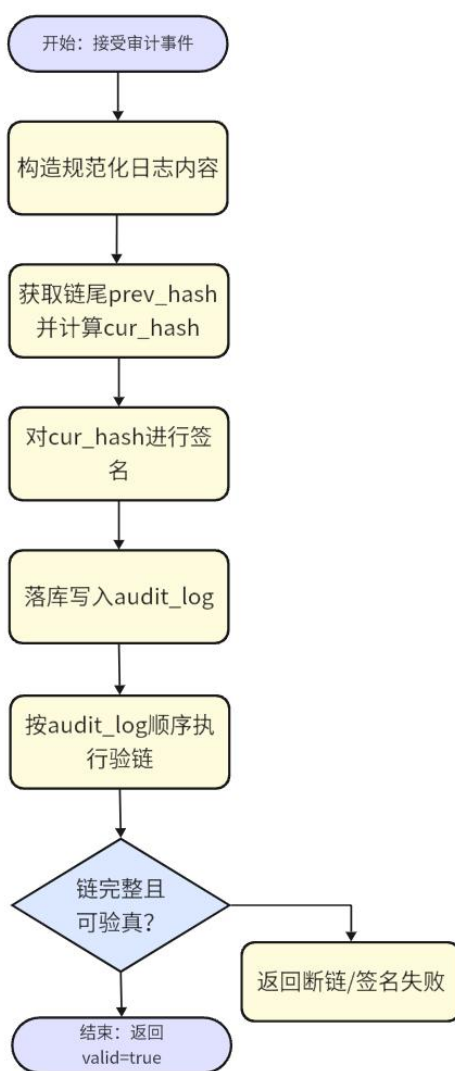


图 3-11 审计链生成与验链框架图

本模块采用“调度与审计并行、控制与验证同源”的实现策略：调度侧通过有界队列与并发阈值保障执行秩序，通过状态机推进保障过程可控；审计侧通过链式哈希与签名保障记录可信，通过验链接口保障结果可核。两者协同后，系统不仅具备任务执行能力，还具备监管可见性与事后举证能力。

因此，任务调度与全生命周期审计模块在体系中承担“管控中枢”角色：其价值不只体现在提升运行效率，更在于将机密计算过程转化为可审、可验、可追责的治理对象，为高价值数据场景落地提供制度化技术支撑。

3.3 数据库设计

“信创密舱·TEE 可信计算平台”采用 MySQL 作为核心关系数据库，数据库名为 security_audit，统一使用 utf8mb4 字符集。数据库设计围绕平台的业务流程与可信审计需求展开，形成了“用户与账号管理—任务管理—TEE 资源管理—远程证明管理—

表 3-1 用户表

编号	字段名	数据类型	注释	约束关系
1	user_id	BigInt Unsigned	用户 ID	PK, 自增
2	username	Varchar(64)	用户名	UNIQUE, NOT NULL
3	password_hash	Varchar(255)	密码哈希	NOT NULL
4	real_name	Varchar(100)	真实姓名	可空
5	email	Varchar(100)	邮箱	可空
6	phone	Varchar(20)	手机号	可空
7	created_at	Datetime	创建时间	默认当前时间
8	updated_at	Datetime	更新时间	默认当前时间, 更新自动刷新

(2) 账号表 (accounts)：账号表用于承载业务操作主体，记录账号状态、登录信息以及所属用户关系。具体设计如表 3-2 所示：

表 3-2 账号表

编号	字段名	数据类型	注释	约束关系
1	account_id	BigInt Unsigned	账号 ID	PK, 自增
2	user_id	BigInt Unsigned	所属用户 ID	FK→users.user_id, NOT NULL, ON DELETE CASCADE
3	account_name	Varchar(128)	账号名称	NOT NULL
4	is_logged_in	TinyInt(1)/Boolean	是否登录	NOT NULL, 默认 0
5	last_login_ip	Varchar(45)	最近登录 IP	可空

编号	字段名	数据类型	注释	约束关系
6	last_login_time	Datetime	最近登录时间	可空
7	status	TinyInt(1)/Boolean	账号状态	NOT NULL，默认 1
8	created_at	Datetime	创建时间	默认当前时间

(3) 任务表 (tasks)：任务表是平台业务流程的核心索引表，记录任务 ID、任务名称以及任务归属账号。具体设计如表 3-3 所示：

表 3-3 任务表

编号	字段名	数据类型	注释	约束关系
1	task_id	Varchar(36)	任务唯一编号 (UUID)	PK
2	account_id	BigInt Unsigned	所属账号 ID	FK→accounts.account_id, NOT NULL, ON DELETE CASCADE
3	task_name	Varchar(255)	任务名称	NOT NULL

(4) 安全审计日志表 (audit_log)：审计日志表用于记录平台关键操作全链路数据，是可信审计能力的核心落地表。具体设计如表 3-4 所示：

表 3-4 安全审计日志表

编号	字段名	数据类型	注释	约束关系
1	log_id	BigInt Unsigned	日志 ID	PK，自增
2	event_time	Datetime(6)	事件时间 (微秒)	NOT NULL，默认当前时间
3	operation_type	Varchar(64)	操作类型	NOT NULL



编号	字段名	数据类型	注释	约束关系
4	details	Text	操作详情	NOT NULL
5	account_id	BigInt Unsigned	操作账号 ID	FK→accounts.account_id, NOT NULL, ON DELETE RESTRICT
6	container_id	Varchar(64)	容器 ID	可空
7	task_name	Varchar(255)	任务名称	NOT NULL
8	task_id	Varchar(36)	任务 ID	NOT NULL
9	status	Enum('SUCCESS','FAILURE')	操作结果	NOT NULL
10	trust_evidence	Varchar(512)	可信证据	可空
11	cur_log_hash	Char(64)	当前日志哈希	NOT NULL
12	prev_log_hash	Char(64)	前序日志哈希	NOT NULL
13	log_signature	Varchar(512)	日志签名	NOT NULL
14	source_ip	Varchar(45)	来源 IP	NOT NULL

(5) 用户通知表 (user_notifications)：用户通知表用于保存用户端消息通知数据，支持通知标题、内容、已读状态与时间维度管理。具体设计如表 3-5 所示：

表 3-5 用户通知表

编号	字段名	数据类型	注释	约束关系
1	notification_id	Varchar(64)	通知 ID	PK



编号	字段名	数据类型	注释	约束关系
2	account_id	BigInt Unsigned	所属账号 ID	FK→accounts.account_id, NOT NULL, ON DELETE CASCADE
3	title	Varchar(255)	通知标题	NOT NULL
4	content	Text	通知内容	可空
5	is_read	TinyInt(1)/Boolean	是否已读	NOT NULL, 默认 0
6	created_at	Datetime	通知时间	NOT NULL, 默认当前时间

(6) 任务扩展信息表 (task_details)：任务扩展信息表用于保存任务过程中的业务上下文，与任务表一一对应。具体设计如表 3-6 所示：

表 3-6 任务扩展信息表

编号	字段名	数据类型	注释	约束关系
1	task_id	Varchar(36)	任务 ID	PK, FK→tasks.task_id, ON DELETE CASCADE
2	scene	Varchar(64)	任务场景	可空
3	algorithm	Varchar(128)	算法名称	可空
4	params_json	Text	训练参数 (JSON)	可空
5	dataset_json	Text	数据集信息 (JSON)	可空
6	status	Varchar(32)	任务状态	NOT NULL, 默认 running



编号	字段名	数据类型	注释	约束关系
7	created_at	Datetime	创建时间	默认当前时间
8	finished_at	Datetime	完成时间	可空
9	timeline_json	Text	任务时间线 (JSON)	可空
10	result_files_json	Text	结果文件列表 (JSON)	可空
11	epc_memory_mb	Integer	EPC 占用 (MB)	可空
12	node_name	Varchar(128)	节点名称	可空
13	container_name	Varchar(128)	容器名称	可空
14	updated_at	Datetime	更新时间	默认当前时间，更新自动刷新

(7) 任务审计事件表 (task_audit_events)：用于记录任务执行过程中的阶段性事件，支撑任务级审计时间线展示与复盘。具体设计如表 3-7 所示：

表 3-7 任务审计事件表

编号	字段名	数据类型	注释	约束关系
1	event_id	Varchar(64)	事件 ID	PK
2	task_id	Varchar(36)	任务 ID	FK→tasks.task_id, NOT NULL, ON DELETE CASCADE
3	event_time	Datetime	事件时间	NOT NULL，默认当前时间
4	event_type	Varchar(64)	事件类型	NOT NULL



编号	字段名	数据类型	注释	约束关系
5	result	Varchar(32)	事件结果	NOT NULL，默认 success
6	node_name	Varchar(128)	TEE 节点名	可空
7	container_name	Varchar(128)	容器名	可空
8	detail_json	Text	事件详情 (JSON)	可空

(8) TEE 节点池表 (tee_cluster_nodes)：用于维护可信计算节点资源与运行状态，是容器调度与证明关联的基础数据源。具体设计如表 3-8 所示：

表 3-8 TEE 节点池表

编号	字段名	数据类型	注释	约束关系
1	node_id	Varchar(64)	节点 ID	PK
2	node_name	Varchar(128)	节点名称	NOT NULL
3	ip	Varchar(64)	节点 IP	可空
4	cpu_model	Varchar(128)	CPU 型号	可空
5	tee_type	Varchar(32)	TEE 类型	NOT NULL，默认 CSV
6	status	Varchar(32)	节点状态	NOT NULL，默认 online
7	epc_total_mb	Integer	EPC 总量 (MB)	NOT NULL，默认 1024
8	epc_used_mb	Integer	EPC 已用 (MB)	NOT NULL，默认 0
9	csv_protected_size	Integer	CSV 保护区 (MB)	可空



编号	字段名	数据类型	注释	约束关系
10	csv_protection_level	Varchar(32)	CSV 保护级别	可空
11	hardware_json	Text	硬件信息 (JSON)	可空
12	recent_proofs_json	Text	最近证明记录 (JSON)	可空
13	created_at	Datetime	创建时间	默认当前时间
14	updated_at	Datetime	更新时间	默认当前时间，更新自动刷新

(9) 机密容器表 (confidential_containers)：用于记录容器实例信息、节点归属和资源占用数据，支撑运行态可视化与运维管理。具体设计如表 3-9 所示：

表 3-9 机密容器表

编号	字段名	数据类型	注释	约束关系
1	container_id	Varchar(64)	容器 ID	PK
2	container_name	Varchar(255)	容器名称	NOT NULL
3	image	Varchar(255)	镜像地址	NOT NULL
4	status	Varchar(32)	容器状态	NOT NULL，默认 Pending
5	node_id	Varchar(64)	所在节点 ID	FK→tee_cluster_nodes.node_id，可空，ON DELETE SET NULL
6	tee_type	Varchar(32)	TEE 类型	NOT NULL，默认 CSV
7	epc_limit_mb	Integer	EPC 限制	可空



编号	字段名	数据类型	注释	约束关系
8	epc_used_mb	Integer	EPC 已用 (MB)	可空
9	logs_json	Text	容器日志	可空
10	audit_json	Text	容器审计信息 (JSON)	可空
11	created_at	Datetime	创建时间	默认当前时间
12	updated_at	Datetime	更新时间	默认当前时间，更新自动刷新

(10) 管理员审计事件表 (admin_audit_events)：用于管理端对象操作留痕，保障后台治理过程可查询、可追踪。具体设计如表 3-10 所示：

表 3-10 管理员审计事件表

编号	字段名	数据类型	注释	约束关系
1	event_id	Varchar(64)	事件 ID	PK
2	event_time	Datetime	事件时间	NOT NULL，默认当前时间
3	operation_type	Varchar(64)	操作类型	NOT NULL
4	object_type	Varchar(64)	对象类型	NOT NULL
5	object_id	Varchar(128)	对象 ID	NOT NULL
6	actor	Varchar(64)	操作者	NOT NULL，默认 system
7	result	Varchar(32)	操作结果	NOT NULL，默认 success
8	detail_json	Text	事件详情	可空

(11) 远程证明报告表 (attestation_reports)：用于保存节点与任务维度的证明材料及验证结果，是平台可信状态证明的关键数据载体。具体设计如表 3-11 所示：

表 3-11 远程证明报告表

编号	字段名	数据类型	注释	约束关系
1	report_id	Varchar(64)	报告 ID	PK
2	node_id	Varchar(64)	节点 ID	FK→tee_cluster_nodes.node_id, 可空, ON DELETE SET NULL
3	task_id	Varchar(36)	任务 ID	FK→tasks.task_id, 可空, ON DELETE SET NULL
4	report_time	Datetime	报告时间	NOT NULL, 默认当前时间
5	validation_json	Text	验证结果 (JSON)	可空
6	evidence_json	Text	证据数据 (JSON)	可空
7	quote_json	Text	Quote 数据 (JSON)	可空
8	benchmark_json	Text	基准对比 (JSON)	可空

(12) 系统配置表 (system_config)：用于管理平台关键配置项，支持配置键值化存储与统一维护。具体设计如表 3-12 所示：

表 3-12 系统配置表

编号	字段名	数据类型	注释	约束关系
1	config_key	Varchar(128)	配置键	PK
2	config_value	Text	配置值	可空
3	value_type	Varchar(32)	值类型	NOT NULL，默认 string
4	description	Varchar(255)	配置说明	可空
5	updated_by	Varchar(64)	更新人	可空
6	updated_at	Datetime	更新时间	默认当前时间，更新自动刷新

3.4 前端界面

(1) 登录界面：

登录界面作为平台统一身份认证入口，采用账号密码校验与角色权限预分配机制，实现业务用户与系统管理员的分区登录管控。界面集成表单实时校验、错误提示与登录状态锁定功能，完成用户身份合法性验证后，依据角色权限自动跳转至对应操作门户，从入口端保障平台访问安全，为后续可信计算操作提供身份基础。登录界面如图 3-13 所示：

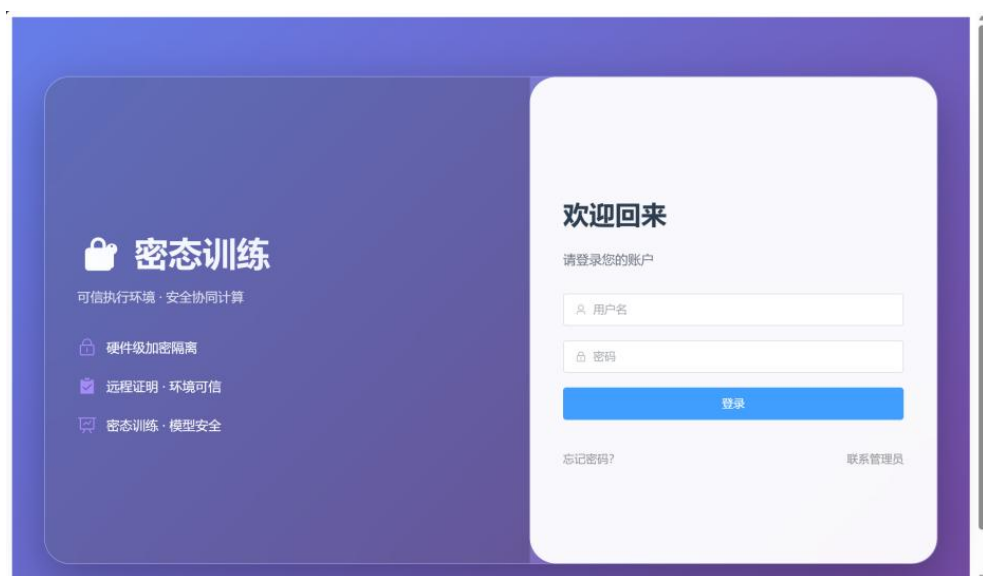


图 3-13 登录界面

(2) 账号看板界面（用户端）：

账号看板为医保协同业务用户提供一站式个人工作台，核心展示用户名下训练任务总览、任务执行状态分布、近期操作记录与待办提醒。界面以可视化卡片形式呈现任务统计数据，支持快速入口跳转至任务创建、结果下载、审计查询等高频功能，简化业务操作路径，同时实时同步任务执行进度与系统通知，便于用户直观掌握个人业务开展状态。用户端账号看板如图 3-14 所示：

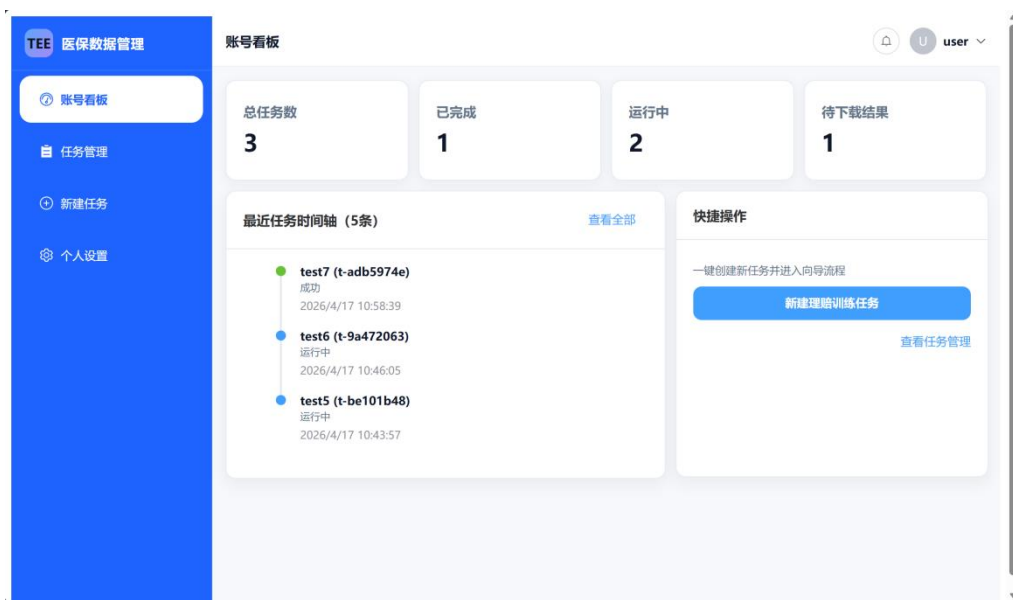


图 3-14 账号看板界面

(3) 任务管理界面（用户端）：

任务管理界面承载业务用户全流程密态训练任务管控，支持训练任务的创建、查询、状态监控、结果导出与删除操作。界面以列表形式展示任务核心信息，包含任务名称、算法类型、创建时间、执行状态、进度百分比等字段，提供筛选、排序、搜索功能，实时同步任务执行日志，用户可清晰追踪任务从创建到完成的全生命周期状态，实现医保数据密态计算任务的便捷管理。用户端任务管理界面如图 3-15 所示：



图 3-15 (1) 任务管理界面

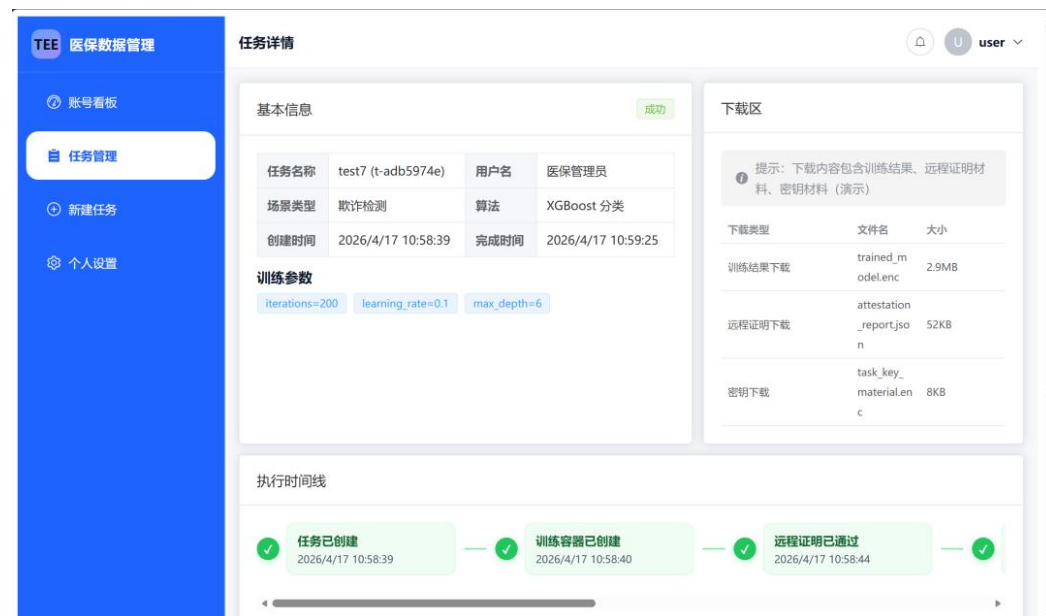


图 3-15 (2) 任务管理界面

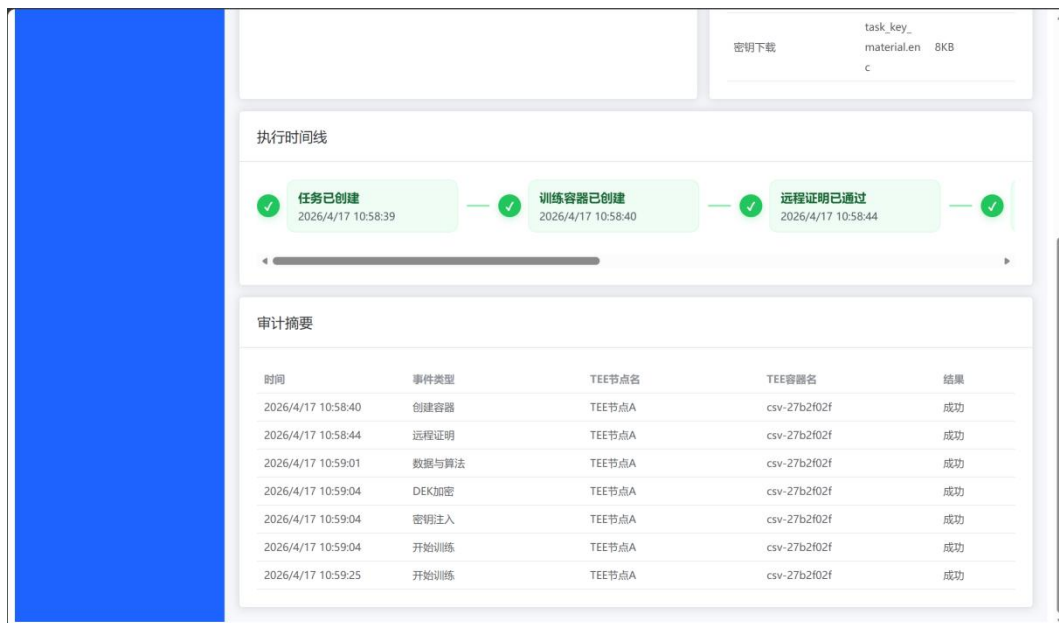


图 3-15 (3) 任务管理界面

(4) 新建任务界面（用户端）：

新建任务界面为用户提供标准化密态训练任务配置入口，支持选择医保理赔适配算法、上传加密训练数据、配置训练超参数、设定样本规模阈值等操作。界面内置参数校验与格式提示功能，自动关联远程证明、密钥注入等前置流程，完成配置后一键提交任务，无需用户处理 TEE 底层逻辑，降低密态计算任务创建门槛，保障任务配置合规性与准确性。新建任务界面如图 3-16 所示：



图 3-16 (1) 新建任务界面



图 3-16 (2) 新建任务界面

(5) 个人设置界面（用户端）：

个人设置界面面向业务用户提供账号信息管理与安全配置功能，支持修改登录密码、更新个人联系方式、查看登录日志、设置消息通知偏好等操作。界面采用密码二次验证、信息修改留痕机制，保障账号信息安全，同时提供操作指引与帮助入口，方便用户自主维护账号状态，提升平台使用便捷性与安全性。个人设置界面如图 3-17 所示：

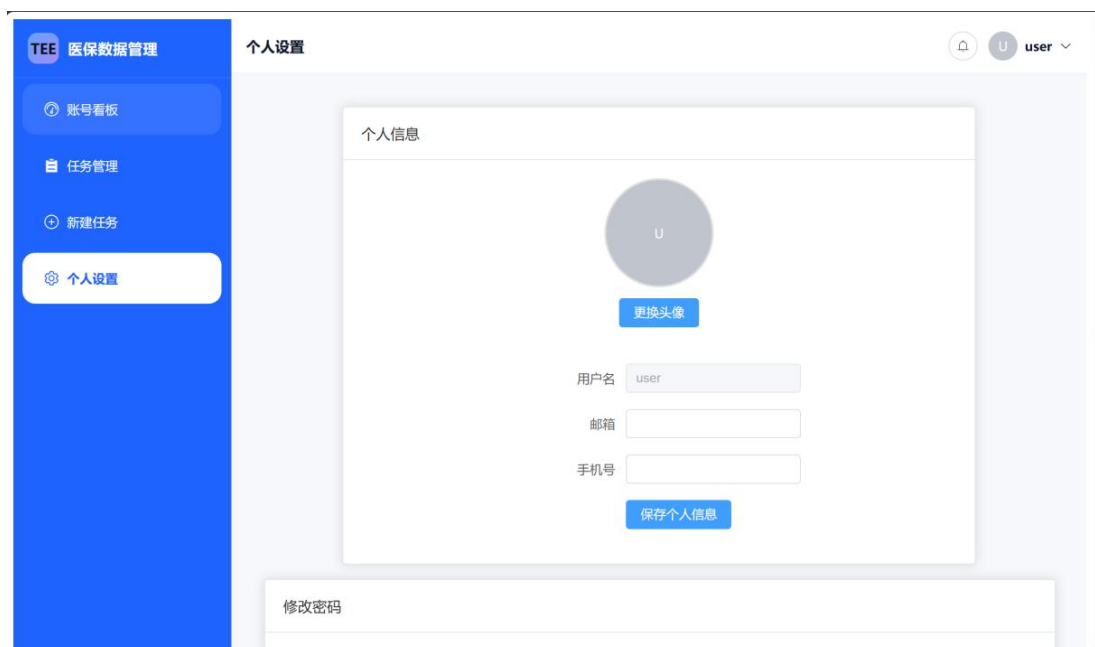


图 3-17 (1) 个人设置界面

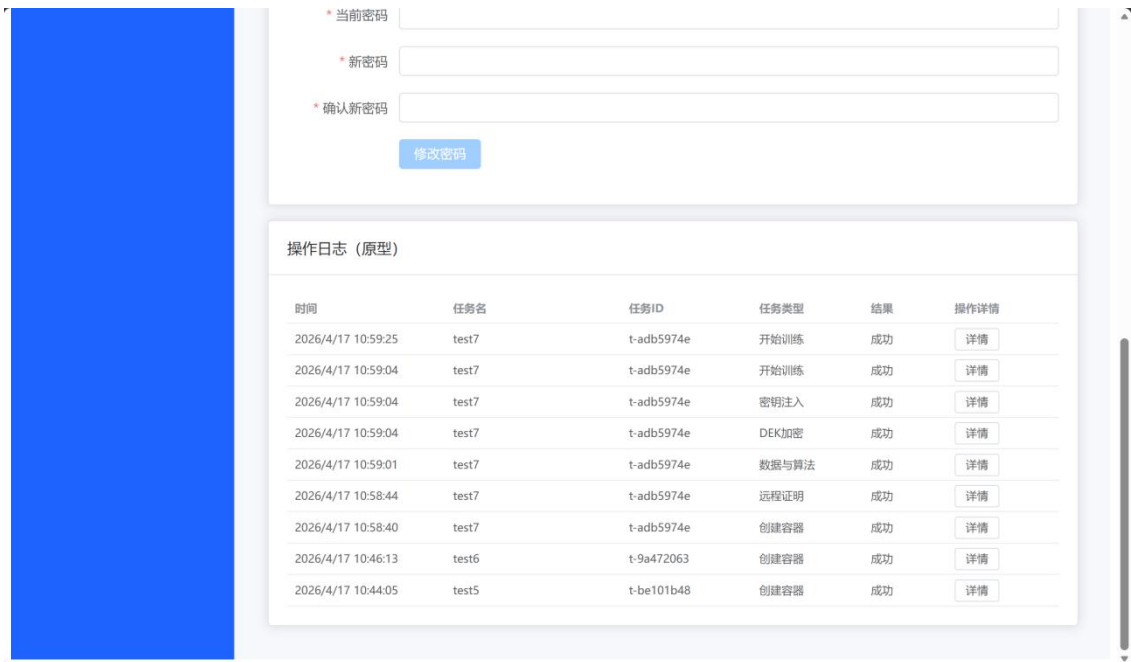


图 3-17 (2) 个人设置界面

(6) 仪表盘界面（管理员端）：

仪表盘作为系统管理员总控门户，集成平台核心运行指标可视化展示，包含机密容器运行状态分布、TEE 节点资源使用率、任务并发量、审计事件统计、系统异常告警等数据。界面通过数据卡片等形式实时呈现系统运行态势，支持指标钻取查看详情，为管理员提供全局运维监控视角，快速定位系统运行风险与资源瓶颈。管理员端仪表盘界面如图 3-18 所示：



图 3-18 仪表盘界面

(7) TEE 资源池界面（管理员端）：

TEE 资源池界面实现国产海光 CSV 可信硬件节点的集中化管理，展示节点 IP、CPU 型号、TEE 类型、EPC 内存总量与使用率、节点在线状态、硬件证书信息等核心参数。支持节点状态监控、资源用量统计、远程证明记录查询功能，管理员可实时掌握可信硬件资源分配与运行情况，保障机密计算硬件底座稳定可用。管理员端 TEE 资源池界面如图 3-19 所示：



图 3-19 (1) TEE 资源池界面



图 3-19 (2) TEE 资源池界面

(8) 机密容器管理页面（管理员端）：

机密容器管理页面负责平台所有机密容器的全生命周期管控，展示容器 ID、镜像哈希、运行状态、所属节点、资源占用、创建时间等信息，支持容器创建、销毁、状态

刷新、日志查看操作。界面实时同步容器运行数据，校验镜像可信性与环境安全性，管理员可直观管控容器运行状态，及时清理异常容器，保障密态计算环境合规与资源高效利用。机密容器管理界面如图 3-20 所示：



图 3-20（1） 机密容器管理页面



图 3-20（2） 机密容器管理页面

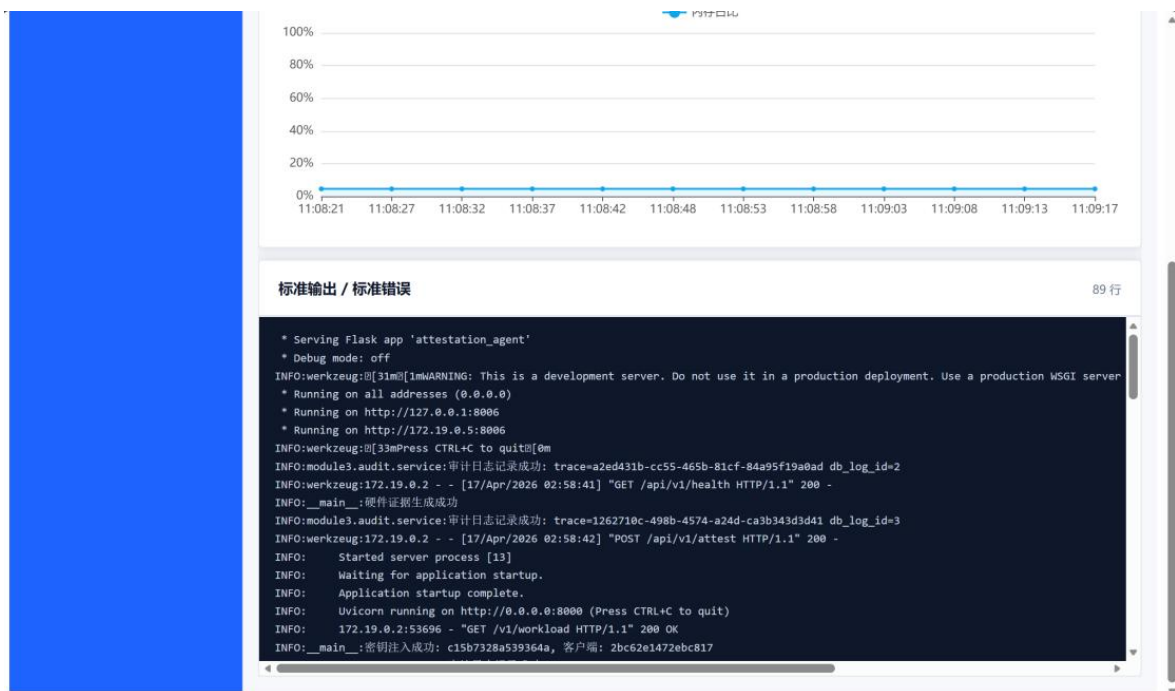


图 3-20 (3) 机密容器管理页面

(9) 审计日志界面（管理员端）：

审计日志界面承载平台全流程操作追溯与合规审计功能，记录用户操作、任务执行、环境验证、密钥注入、容器调度等关键行为，包含操作时间、账号信息、任务 ID、操作结果、来源 IP、可信证据等字段。界面支持多维度筛选、日志导出、哈希验真操作，日志采用链式哈希与国密签名保障不可篡改，满足医保行业合规审计与责任追溯要求。审计日志界面如图 3-21 所示：

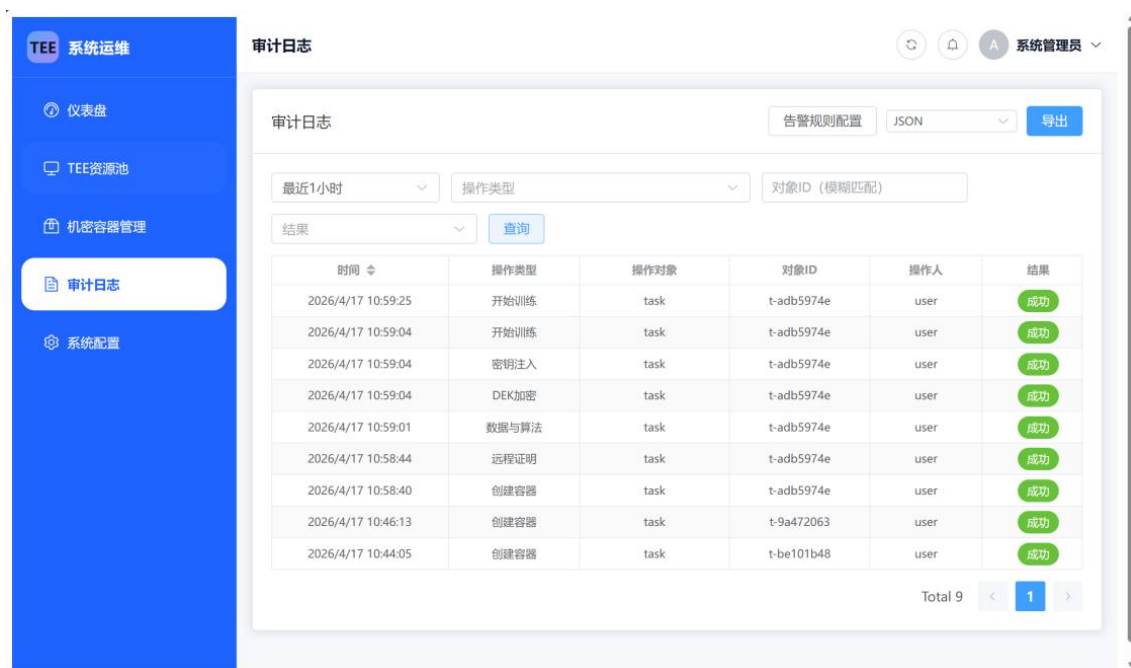


图 3-21 审计日志界面

(10) 系统配置页面（管理员端）：

系统配置页面为管理员提供平台全局参数管控功能，支持可信镜像白名单管理、并发任务阈值设置、密钥过期时间配置、审计日志保存周期调整、国密算法参数设定等操作。配置项与底层机密计算逻辑联动，修改后实时生效，同时记录配置变更审计日志，保障平台运行参数合规、安全、适配医保业务场景需求。系统配置界面如图 3-22 所示：

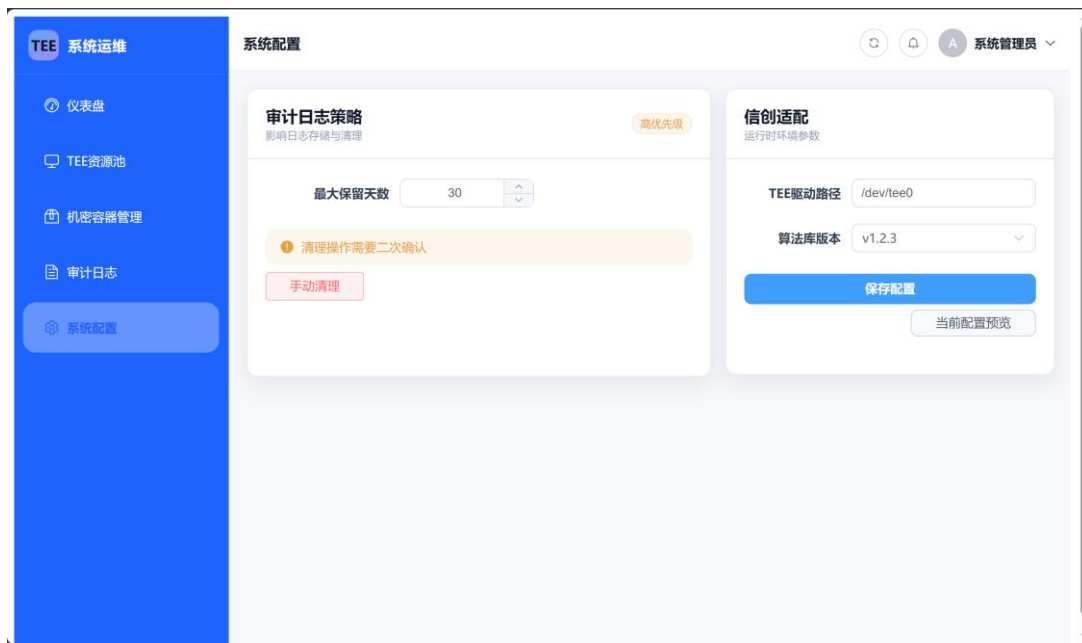


图 3-22 系统配置页面

4 安全保障机制

本项目围绕医保理赔联合建模场景，构建了“准入可控、过程可证、数据可护、操作可审”的安全体系。系统以海光 CSV 机密计算环境为可信执行基础，在任务启动前完成镜像准入校验，在任务执行前完成远程证明与密钥注入，在任务全流程中保留可验证审计记录，形成从任务创建到结果下载的闭环安全链路。

4.1 可信执行环境与准入控制

平台在创建训练容器前，先获取目标镜像哈希，并与证明服务维护的可信白名单进行比对^[12]。镜像哈希未命中白名单时，系统直接拒绝容器创建请求，阻断不可信镜像进入训练平面。该机制将安全校验前置到容器生命周期起点，降低运行后再处置的风险。

在证明机制上，证明代理优先生成 type=csv 的硬件证据；无 CSV 设备时默认不通过。仅在显式开启开发开关的场景下，系统才允许模拟证据路径用于调试联测。证明服务除签名校验外，同时执行证据时效与重复提交校验：超时证据与重复报告均会被拒绝。通过上述策略，平台实现了“镜像可信、环境可信、证据可信”的联合约束。

图 4-1 展示了项目训练信任链的完整路径，覆盖策略配置、镜像准入、远程证明、密钥注入与密态执行等关键环节。详细设计如表 4-1：

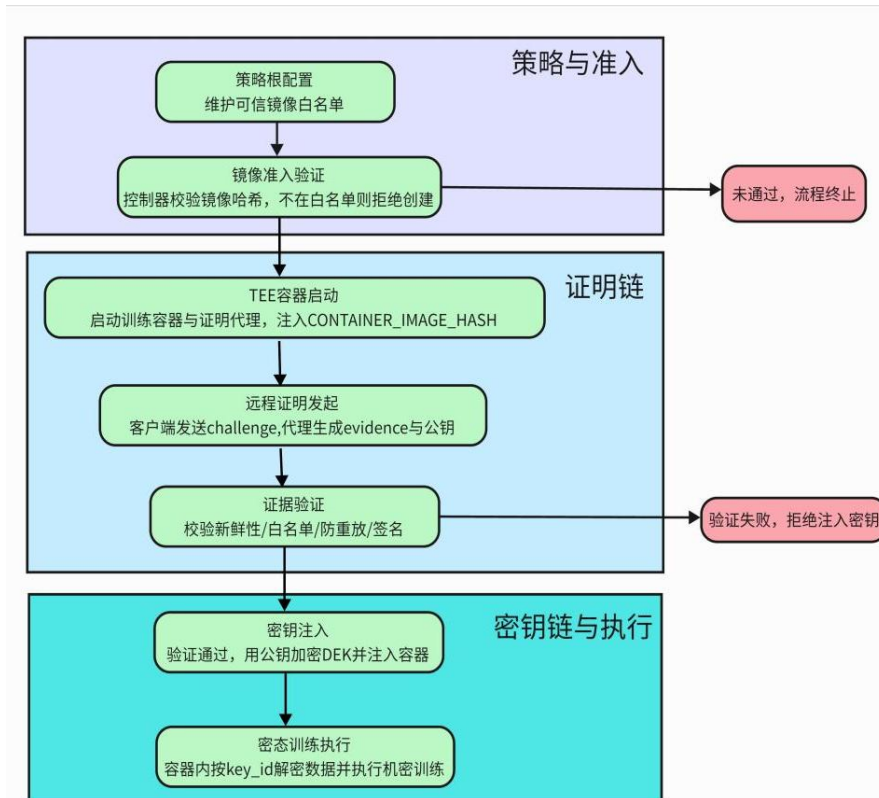


图 4-1 训练信任链与密钥链路

表 4-1 可信环境与准入机制

安全机制	项目实施方式	保护对象	主要风险覆盖
镜像白名单准入	创建容器前校验镜像哈希，仅白名单可创建	训练运行环境	恶意镜像、镜像篡改
真实 CSV 证明优先	默认要求 type=csv 证据，模拟证据需显式开关	计算节点可信性	非真实 TEE 环境接入
证据验签与时效、防重放	验证签名、时间窗口和重复提交	证明链路完整性	伪造证据、重放证明

4.2 传输与密钥安全

本项目传输安全以“关键动作按安全顺序执行”为核心。客户端先发起挑战值请求，证明代理生成证据与公钥，验证服务完成校验；校验失败时，流程在证明阶段终止，不进入密钥注入与训练阶段。

客户端在接收证明响应后，对返回公钥进行哈希计算，并与证据中的公钥哈希字段核对。若二者不一致，系统立即终止后续流程。该校验用于确保“证明证据与注入公钥”保持一致，防止公钥替换风险。

密钥管理采用任务级 DEK 方案：平台侧生成会话密钥，通过证明代理公钥加密后注入；训练服务仅通过 key_id 在会话上下文内调用密钥能力。注入成功是训练任务提交的前置条件。训练数据与模型产物在链路中均通过 SM4 处理，实现“先证明、后授钥、再计算”的顺序闭环。详细设计如表 4-2 所示：

表 4-2 传输与密钥保护机制

安全机制	项目实施方式	保护对象	主要风险覆盖
挑战值证明握手	客户端发起挑战，代理生成证据，验证服务校验	证明会话	非法会话伪造
公钥哈希一致性	本地计算公钥哈希并与证据字段比对	密钥注入目标	公钥替换、中间人劫持
DEK 注入后置训练	注入 key_id 成功后才允许提交训练任务	数据与模型密钥	未授权解密、流程绕过

4.3 数据与存储安全

数据进入平台后先进行加密，再以密文文件参与训练流程。项目提供了配套验证路径，可证明密文文件无法被直接按明文表读取，错误密钥也无法还原可用训练数据，满足“TEE 外不落明文”的交付要求。

训练侧按任务上下文调用解密能力，解密请求必须携带有效 `key_id`，并与任务流程绑定执行。系统未提供独立的开放式明文导出链路，减少明文在非必要模块中的扩散范围。

任务完成后，模型结果以加密文件形式输出和下载，与训练数据的密态处理形成前后一致的保护路径，确保输入和输出两端均处于受控状态。详细设计如表 4-3：

表 4-3 数据与存储保护机制

安全机制	项目实现方式	保护对象	主要风险覆盖
数据加密落盘	上传后生成密文文件参与训练	原始理赔数据	磁盘泄露、越权读取
<code>key_id</code> 绑定解密	解密需携带有效会话密钥标识	训练期明文数据	非授权解密调用
结果加密下载	训练结果以加密模型文件输出	模型参数与结果文件	结果外泄、链路窃取

4.4 审计与追溯安全

本项目审计体系由业务审计与技术审计共同组成。业务审计用于呈现任务关键步骤，技术审计用于保留接口调用与执行上下文。核心审计记录采用链式哈希与签名机制，支持完整性校验与事后验真。

审计写入过程中，系统读取上一条哈希并计算当前哈希，形成连续链路。针对跨服务并发写入场景，项目通过数据库全局锁保证链尾一致性，避免并发导致链路分叉。平台提供验链接口，可输出校验报告，支持现场核验。

审计记录绑定任务、容器、节点和操作者信息。出现证明失败、训练失败、下载失败等异常时，可在任务视图与审计视图中同步定位对应节点，支撑快速复盘与责任追踪。详细设计如表 4-4：



表 4-4 审计与治理机制

安全机制	项目实施方式	保护对象	主要风险覆盖
日志哈希链 + 签名	日志前后哈希关联并支持验链	审计记录完整性	日志篡改、删除后抵赖
任务 - 容器 - 操作者关联	审计记录绑定任务与执行主体	操作可追溯性	责任不可归因
异常双通道留痕	任务视图与审计视图同步记录失败事件	运维处置效率	异常发现滞后

5 性能优化策略

本项目在机密计算场景下的性能优化目标，是在安全机制完整保留的前提下，降低固定开销、压缩链路时延、提升并发稳定性。结合当前实现，性能影响主要集中在三类环节：容器准备与远程证明、加解密与数据处理、任务调度与状态轮询。

5.1 证明链路与准入效率优化

控制器默认从证明服务拉取可信镜像白名单；在证明服务短时不可达或超时场景下，系统可按本地可信副本继续执行准入判断，降低无效失败重试对任务时延的影响。该机制保持准入标准一致，不放宽安全边界。

向导流程在发起远程证明前执行健康探测等待，待证明代理就绪后再提交挑战请求，减少启动阶段连接失败与重复请求开销，提高首轮证明成功率和流程稳定性。

5.2 训练链路与容器生命周期优化

动态训练容器内同时运行证明代理与训练服务，密钥注入和训练调用在同一实例内完成，减少跨容器调用带来的额外网络跳转。该部署方式在保证流程隔离的同时，降低链路时延和故障定位复杂度。

控制器内置空闲回收机制，根据空闲阈值与最小存活时间回收训练容器，减少长期空转占用。训练完成后系统可刷新容器活跃时间，为结果查询与下载保留必要窗口，在资源利用率与可达性之间保持平衡。

5.3 调度并发与系统吞吐优化

调度服务采用“队列+并发上限”机制管理训练任务。提交请求先进入队列，再按并发容量进入执行阶段；并发阈值支持动态调整，便于在不同资源条件下维持稳定吞吐，避免高峰期资源争抢导致级联超时。

训练任务采用异步执行模式：接口在任务创建后快速返回任务标识，执行进度通过状态查询获取。该方式缩短前台请求等待时间，提升多任务并行场景下的整体响应能力。

5.4 管理面采样与轮询优化

管理端节点监控采用短周期缓存，减少高频重复采集带来的附加负载。容器状态查询同样采用状态缓存机制，降低管理面轮询对核心服务的干扰，提升可视化页面稳定性与刷新流畅性。

5.5 基准评测口径与复现方式

项目提供明文路径与“SM4 加解密+训练+模型加解密”路径的对照基准，统一输出训练耗时中位数、相对损耗比例和结果一致性差值。该口径用于支撑比赛材料中的性能结论与一致性结论。

建议在同机同配置条件下进行多轮测试，采用中位数作为统计值；端到端评测时将容器创建、证明等待、网络传输等因素纳入总耗时，形成完整性能说明。优化措施与作用点详细设计如表 5-1 所示：

表 5-1 优化措施与作用点

优化措施	作用环节	项目实现路径	预期效果
白名单容错校验	准入阶段	证明服务优先 + 本地白名单回退	降低外部依赖抖动导致的失败重试
证明前健康探测	证明阶段	向导侧代理就绪等待	提升首轮证明成功率
同容器链路执行	训练前后处理	代理与训练服务同实例协同	减少链路跳转与通信损耗
空闲回收策略	资源治理	控制器空闲检测与自动回收	提升资源利用率，避免长期空转
队列化并发调度	系统吞吐	调度队列 + 并发上限动态控制	提升并发稳定性与可预测性
异步任务执行	交互体验	提交即返回、后台训练、状态轮询	改善接口时延与多任务体验
指标与状态短缓存	管理面性能	节点指标缓存与状态缓存	降低高频查询开销，提升可视化流畅度

6 API 接口

6.1 接口设计原则

本章面向“基于机密容器的可信协同计算平台”给出统一 API 规范，覆盖认证鉴权、可信验证、密钥管理、密态训练、监控审计五类核心能力。平台用户角色仅包含两类：用户与管理员。其中，用户负责数据与任务主链路操作，管理员负责资源治理、安全审计与策略管理。

接口体系遵循四项原则。第一，安全优先。所有接口统一采用 HTTPS 传输，结合 JWT 令牌鉴权和关键操作审计留痕，保障调用过程可控、可追踪。第二，能力抽象。平台对底层 TEE 细节进行统一封装，调用方无需关注硬件差异与底层证明格式，实现“同一接口、多环境适配”。第三，异步解耦。针对远程证明、密钥注入、密态训练等耗时流程，采用异步任务机制，避免同步阻塞导致系统吞吐下降。第四，全链路可审计。所有关键请求强制携带请求追踪标识，支持任务级、用户级、时间级多维度溯源，满足比赛场景下对安全可验证性的要求。

API 分层与调用关系如图 6-1 所示：

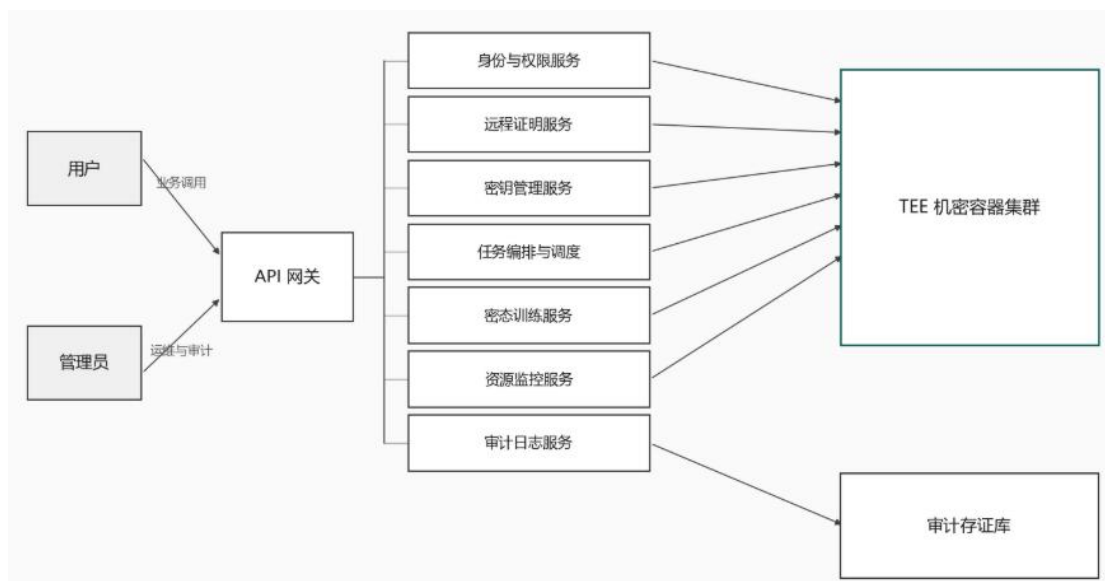


图 6-1 API 分层与调用关系图

6.2 接口清单总览

为形成完整业务闭环，平台定义 16 个核心接口，覆盖“认证鉴权—可信验证—密钥与数据—任务执行—监控审计”五大模块。接口设计不追求数量最小化，而强调环节完整性与可验证性，确保从任务开始到结果输出全程可控、可追踪、可复核。详细 API 设计如表 6-1 所示：



表 6-1 核心 API 总览表

编号	接口名称	方法	路径	功能说明	主要调用角色
API-01	获取访问令牌	POST	/api/v1/auth/token	登录并签发短期令牌	用户 / 管理员
API-02	刷新访问令牌	POST	/api/v1/auth/refresh	刷新会话令牌	用户 / 管理员
API-03	查询 TEE 能力清单	GET	/api/v1/attestation/capabilities	查询节点 TEE 能力与算法支持	管理员
API-04	发起远程证明挑战	POST	/api/v1/attestation/challenge	生成挑战值并绑定任务上下文	用户 / 管理员
API-05	提交证据并验证	POST	/api/v1/attestation/verify	校验证据、度量值、镜像摘要	用户 / 管理员
API-06	建立安全会话	POST	/api/v1/secure-channel/session	基于证明结果建立安全信道	用户
API-07	申请数据密钥 DEK	POST	/api/v1/keys/dek/apply	申请任务级一次性密钥	用户
API-08	向 TEE 注入密钥	POST	/api/v1/keys/inject	将密钥定向注入机密容器	用户 / 管理员
API-09	上传加密数据分片	POST	/api/v1/data/chunks/upload	分片上传密文数据并校验完整性	用户
API-10	提交数据装载任务	POST	/api/v1/data/load-jobs	在 TEE 内完成解封与装载	用户
API-11	创建密态训练任务	POST	/api/v1/jobs/train	提交算法、数据、资源规格并调度	用户
API-12	查询任务状态	GET	/api/v1/jobs/{jobId}	查询生命周期与执行进度	用户 / 管理员

编号	接口名称	方法	路径	功能说明	主要调用角色
API-13	取消训练任务	POST	/api/v1/jobs/{jobId}/cancel	安全终止任务并清理上下文	用户 / 管理员
API-14	导出加密结果	POST	/api/v1/results/export	导出密文模型与结果包	用户
API-15	查询 TEE 资源监控	GET	/api/v1/metrics/tee	查询 CPU / 内存 / 队列等指标	管理员
API-16	查询安全审计日志	GET	/api/v1/audit/events	查询关键安全事件与操作记录	管理员

6.3 核心接口说明

本节按实际调用链路给出接口功能说明，强调“证明先行、密钥后置、任务受控、结果密态导出”的执行逻辑。

(1) 认证与会话接口

API-01 用于令牌签发，返回访问凭据与角色信息，是后续所有接口调用前提。API-02 用于令牌续期，降低长期凭据泄露风险。认证失败场景纳入限流与告警规则，避免暴力尝试。

(2) 可信验证接口

API-03 由管理员用于查看 TEE 基础能力，支撑调度准入。API-04 生成一次性挑战值（nonce），防止重放。API-05 提交并校验证据，校验项包括硬件证明报告、启动度量值与镜像摘要白名单。API-06 在验证通过后建立安全会话，为后续密钥传输提供机密通道。该阶段决定任务是否进入执行流程，是平台可信闭环的核心关口。

(3) 密钥与数据接口

API-07 按任务粒度申请 DEK，避免长期静态密钥。API-08 将密钥与受信会话绑定并注入目标 TEE 实例，确保密钥仅在可信环境可用。API-09 支持密文数据分片上传与完整性校验，解决大样本数据集传输可靠性问题。API-10 触发 TEE 内解封和装载，形成训练输入视图。该阶段保证“数据与密钥在 TEE 外始终保持密态”。

(4) 训练与导出接口

API-11 创建密态训练任务，绑定算法类型、镜像摘要与资源配额。API-12 提供任务状态查询能力，支持业务侧进行流程编排。API-13 在策略变更或异常情况下实现安全终止和上下文清理。API-14 将训练结果按密文形式导出，附带校验值供接收方验真，避免明文模型外泄。

(5) 监控与审计接口

API-15 用于 TEE 资源监控，重点输出性能损耗评估所需指标。API-16 用于审计日志检索，覆盖认证、证明、密钥调用、任务调度、权限变更等关键事件，为赛题中的“全生命周期审计”提供证据基础。

统一响应结构如表 6-2 所示：

表 6-2 统一响应结构

字段名	类型	说明
code	string	业务状态码，0 表示成功
message	string	返回说明（成功 / 失败描述信息）
requestId	string	请求追踪 ID，用于问题排查与链路追踪
timestamp	string	服务端时间戳
data	object	业务返回体，不同接口返回不同结构数据

6.4 异步调用机制

由于远程证明、密钥注入、数据装载与训练执行均为长流程，本平台统一采用异步机制。调用方发起请求后即时获取受理结果和任务 ID，后续通过 API-12 轮询状态或接回收调消息。异步机制显著降低前端等待时间，提高系统并发吞吐能力，并将安全校验与业务执行进行解耦。

状态机定义为 PENDING、RUNNING、SUCCEEDED、FAILED、CANCELLED 五种状态。所有写接口支持幂等键，确保网络重试不产生重复任务。回调消息包含签名与时间戳，保障消息来源可信与消息不可重放。

任务状态机说明如表 6-3：

表 6-3 任务状态机说明

状态	含义	是否终态
PENDING	已受理，等待资源调度	否
RUNNING	已进入执行阶段	否

状态	含义	是否终态
SUCCEEDED	执行成功	是
FAILED	执行失败	是
CANCELLED	人工或策略取消	是

6.5 错误码与安全规范

接口错误码采用“可定位、可处置”原则。认证失败、权限不足、证明失败、完整性校验失败、密钥注入失败、训练执行异常等场景均有独立编码，便于快速定位根因并触发自动化处置流程。

安全规范采用五层防护：认证层（JWT）、授权层（RBAC）、传输层（HTTPS）、机密层（密钥与数据密态）、审计层（全链路留痕）。对高危事件系统强制写审计并触发告警，满足比赛对“抗宿主机侧攻击与操作可追责”的要求。

关键错误代码如表 6-4：

表 6-4 关键错误码示例

错误码	含义	处理建议
E40101	令牌无效或过期	重新登录或刷新令牌
E40310	权限不足	检查用户角色与策略绑定
E40920	远程证明失败	复核节点可信状态与镜像基线
E40921	度量值不匹配	更新白名单或更换受信节点
E42230	数据完整性校验失败	重新上传错误分片
E50041	密钥注入失败	检查会话与 DEK 有效性
E50050	训练执行异常	查看任务日志与监控指标

6.6 典型调用流程

以“医保理赔联合建模”场景为例，平台完整业务链路如下：

第一步，用户调用认证接口完成登录并获取令牌。第二步，用户发起远程证明挑战并提交证据，平台完成可信校验后建立安全会话。第三步，用户申请 DEK 并向 TEE 注入密钥，随后上传密文数据并提交装载任务。第四步，用户创建密态训练任务，调度服务在受信 TEE 实例内执行训练。第五步，用户通过状态接口持续查询执行进度，任务完成后导出加密模型结果。第六步，管理员查询审计日志与资源监控，完成任务验收与安全复核。该流程实现了“可信验证—密钥闭环—密态计算—加密输出—审计追踪”的端到端安全闭环，能够直接对应赛题验收要求。

端到端调用时序如图 6-2，典型流程与接口映射如表 6-5 所示：

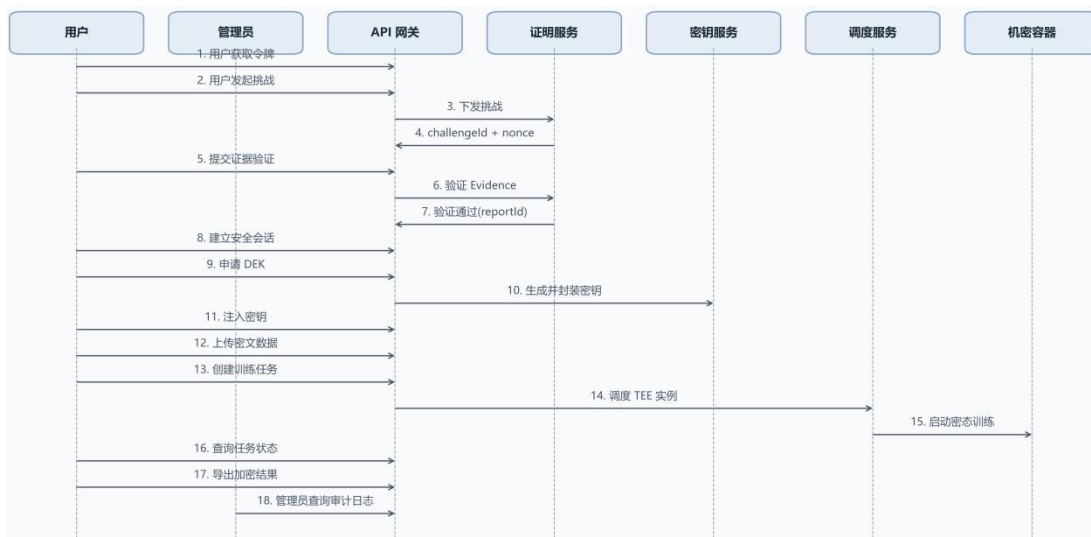


图 6-2 端到端调用时序图

表 6-5 典型流程与接口映射

流程阶段	对应接口
登录与鉴权	API-01、API-02
远程证明	API-03、API-04、API-05、API-06
密钥与数据准备	API-07、API-08、API-09、API-10
密态训练执行	API-11、API-12、API-13
结果输出与监管	API-14、API-15、API-16

7 系统测试

7.1 测试方案

本章测试围绕机密计算协同平台的三类目标展开：功能可用性、安全可靠性与性能可接受性。测试对象覆盖平台关键链路，包括认证鉴权、远程证明、容器创建与管理、密钥注入、密态训练、结果导出与审计追踪，确保系统在“可跑通、可防护、可度量”三个层面形成闭环。

测试环境采用阿里云 ECS 实例，规格为 ecs.g7h.2xlarge，该实例支持相关硬件 TEE 特性。业务场景选取医保理赔联合建模。测试数据按 500、1000、2000 条分级构造，用于观察规模变化下的时延与稳定性。测试执行遵循“先功能后安全再性能”的顺序，减少变量叠加对结果判定的干扰。所有关键步骤均记录接口响应、状态流转和日志证据，确保结果可复核。测试目标与验收口径如表 7-1：

表 7-1 测试目标与验收口径

测试维度	目标	验收口径
功能测试	验证全链路可执行	从认证到结果导出无阻断，状态机流转正确
安全测试	验证攻击防护有效性	关键攻击场景可被识别、阻断，并生成审计记录
性能测试	验证密态开销可控	明文与密态训练开销 < 20%
业务一致性	验证密态训练有效	密态指标与明文基线保持一致性

7.2 功能测试

功能测试依据平台核心业务流程，设计 10 个测试用例，覆盖从用户认证到容器销毁的完整生命周期。每个用例包含明确的测试步骤与验证点，确保平台各模块协同工作正常。功能测试用例及执行结果如表 7-2 所示：

表 7-2 功能测试用例及执行结果

用例编号	测试项	操作步骤	预期结果	实测结论
01	用户认证与授权	1. 提交合法账号密码;2. 获取访问令牌	返回有效 JWT token, 含正确过期时间及权限域	通过
02	容器镜像拉取与校验	1. 指定镜像标签;2. 触发拉取; 3. 校验度量值	拉取成功, SHA256 摘要与预期一致	通过
03	TEE 容器创建与启动	1. 调用容器接口;2. 传入 TEE 参数;3. 等待状态变更	容器创建成功, 状态 CR EATING→RUNNING, 返回 container_id	通过
04	远程证明	1. 发起证明挑战;2. 获取证据; 3. 验证度量值	证据校验通过, app_measurement 匹配, valid=true	通过
05	密钥注入	1. 证明通过后调用密钥接口; 2. 传输密钥材料	密钥注入成功, 返回 key_id, 可用于解密密文	通过
06	密文数据装载	1. 上传加密训练数据;2. 容器解密; 3. 验证明文	解密成功, 解析为 DataFrame, 行数 2001 行	通过
07	密态训练执行	1. 发起训练; 2. 等待完成; 3. 获取日志及产物	任务成功, 生成加密模型及训练指标	通过
08	结果导出	1. 下载加密模型; 2. 解密; 3. 验证加载推理	模型与明文产物一致, 推理正确	通过
09	容器安全销毁	1. 观察任务结束后容器状态; 2. 检查残留	容器销毁, 无敏感数据残留	通过
10	审计追踪	1. 用 request_id 检索日志; 2. 验证完整性	返回全链路审计事件链	通过

全部 10 个功能测试用例均通过验证。平台任务生命周期按预期状态机流转, 关键字段上下文一致, 审计日志可追溯, 功能链路闭环稳定。

测试数据规模与运行镜像检查如图 7-1, 远程证明如图 7-2, 执行结果如图 7-3 所示:



```
[root@izbp1ih5ucl6clquat0cyvZ confidential-computing-platform]# wc -l data/train_claims_numeric.csv
2001 data/train_claims_numeric.csv
[root@izbp1ih5ucl6clquat0cyvZ confidential-computing-platform]# docker images | grep csv-training-service
csv-training-service          latest        68c96ec37460   24 hours ago   1
.49GB
```

图 7-1 测试数据规模与运行镜像检查

```
2026-03-23 14:47:48,813 - module1.client - DEBUG - 验证服务响应: {
  "code": 0,
  "data": {
    "expires_at": 1774252068.811311,
    "measurements": {
      "app_measurement": "479821c2c480726fd15a60666df9c88af3a3d0c4bd594200a2f206753609d4ce",
      "csv_status": "secure",
      "evidence_hash": "c90411ab1a167d34"
    },
    "session_id": "fcfdf60ee5be4056",
    "token": "c90411ab1a167d34edc17267ac6c9c78",
    "valid": true
  }
}
✅远程证明通过
```

图 7-2 远程证明

```
🎉机密计算训练任务执行成功!
[root@izbp1ih5ucl6clquat0cyvZ confidential-computing-platform]# ls -lh data/encrypted_train_data.bin
-rw-r--r-- 1 root root 68K Mar 23 14:47 data/encrypted_train_data.bin
```

图 7-3 执行成功图

7.3 安全测试

安全测试采用“攻击构造—系统响应—日志复核”三段式方法。每个场景同时评估防护效果与审计有效性，确保平台不仅能阻断风险，还能形成可追责证据链。

7.3.1 模拟攻击测试

该测试模拟对密文训练数据的直接读取与错误解码，验证数据在 TEE 外部不可被还原为可用明文。测试中，密文文件呈现随机字节分布，字节均值约 127，无法 pandas 直接解析为业务 CSV；错误密钥解密结果也不满足结构化数据格式要求。结论表明平台可有效抑制宿主机侧的数据直接利用风险。内存嗅探测试结果如表 7-3 所示，密文误解码与不可用性验证如图 7-4 所示，密文文件随机字节观测如图 7-5 所示：

表 7-3 内存嗅探测试结果

检查项	观测结果	判定
密文可读性	非业务可读文本	通过
直接 CSV 解析	解析失败	通过
错误密钥解封	输出不可用数据	通过
明文痕迹检查	未发现关键明文字段	通过

```
[root@iZbplih5ucl6clquat0cyvZ confidential-computing-platform]# python3.8 scripts/verify_ciphertext_and_att
ack_simulation.py \
> --plain data/train_claims_numeric.csv \
> --cipher data/encrypted_train_data.bin \
> --key-hex <5b7a68a324bd2de36a5f54251dala0f3>
-bash: syntax error near unexpected token `newline'
[root@iZbplih5ucl6clquat0cyvZ confidential-computing-platform]# python3.8 scripts/verify_ciphertext_and_att
ack_simulation.py --plain data/train_claims_numeric.csv --cipher data/encrypted_train_data.bin --key-hex 5b
7a68a324bd2de36a5f54251dala0f3
明文 SHA-256: 673a79adebdb87c5c1286112959c570e6144974d8192dd348503084960647f46
密文 SHA-256: b7277edd09a82cf753015cd98f412f4b0526ad79c8c06a864fac6df0574f1457
宿主机直接 pandas.read_csv(密文) 是否成功: False
表头 token 在密文中是否出现(子串): 未发现常见表头明文泄露
错误密钥解密后是否为合法 CSV: False
密文字节均值(约127为随机): 126.93615700092421
✅自检项完成: TEE 外密文不可直接当业务 CSV 使用; 错误密钥不产生可用表。
```

图 7-4 密文误解码与不可用性验证

```
[root@iZbplih5ucl6clquat0cyvZ confidential-computing-platform]# strings data/encrypted_train_data.bin | head -80
je
GX&0eG
wB.:
G
[aELJ
p$Q?
M' H&
/e7z
8t0
P<PC
R[DaP
5-"]
```

图 7-5 密文文件随机字节观测

7.3.2 证书篡改测试

该测试模拟证明证书、度量摘要或相关校验字段被篡改的情况。平台在远程证明环节返回失败，不进入密钥注入与训练流程。审计模块记录失败事件、触发时间与关联任务标识，实现“失败即阻断、异常可追踪”的策略闭环。证书篡改测试结果如表 7-4：

表 7-4 证书篡改测试结果

测试项	预期	实测	判定
篡改证明材料	校验失败	失败并中止	通过
后续任务执行	不允许继续	未进入训练阶段	通过
审计留痕	记录异常事件	已记录	通过

7.3.3 重放攻击测试

测试复用历史挑战值与历史证明报文，验证系统是否具备时效性与唯一性校验。平台对过期、重复上下文请求进行拒绝处理，重放请求无法通过远程证明阶段。相关异常请求在审计日志中具备可检索记录，满足重放防护要求。重放攻击测试结果如表 7-5：

表 7-5 重放攻击测试结果

测试项	预期	实测	判定
重放历史 challenge	拒绝受理	已拒绝	通过
重放历史证明报文	验证失败	已失败	通过
审计检索	可查询重放事件	可查询	通过

7.3.4 中间人攻击测试

该测试模拟链路劫持、报文篡改和伪造会话场景。由于系统采用 TLS 传输保护与证明绑定机制，伪造链路无法建立有效受信会话，关键下发步骤被拒绝。测试结果显示平台具备基本的中间人攻击防护能力。中间人攻击测试结果如表 7-6 所示，安全异常场景下结果失效对比如图 7-6 所示：

表 7-6 中间人攻击测试结果

测试项	预期	实测	判定
伪造证书链路	握手或验证失败	失败	通过
报文篡改	完整性校验失败	失败	通过
劫持后密钥请求	不允许会话建立	拒绝	通过

```

"metrics_plain": {
  "accuracy": 0.94,
  "auc": 0.9575123152709359
},
"metrics_crypto_pipeline": {
  "accuracy": 0.94,
  "auc": 0.9575123152709359
},
"accuracy_abs_diff": 0.0,
"auc_abs_diff": 0.0,
"note": "crypto_pipeline 含: 训练数据 SM4 加解密 + 训练 + 模型 SM4 加解密; 完整机密容器端

```

图 7-6 (1) 安全异常场景下结果失效对比

```

"metrics_plain": {
  "accuracy": NaN,
  "auc": null
},
"metrics_crypto_pipeline": {
  "accuracy": NaN,
  "auc": null
},
"accuracy_abs_diff": NaN,
"auc_abs_diff": null,
"note": "crypto_pipeline 含: 训练数据 SM4 加解密 + 训练 + 模型 SM4 加解密; 完整机密容器端

```

图 7-6 (2) 安全异常场景下结果失效对比

7.4 性能测试

性能测试重点评估密态训练额外开销、不同数据规模下的扩展性及并发请求稳定性。测试口径为相同算力，阿里云 ecs.g7h.2xlarge、相同模型参数 N_EST=8000、相同数据特征条件下对比明文与密态表现，避免因环境差异导致偏差。

7.4.1 明文 vs 密态训练时间对比（2000 条真实数据）

基于截图中的真实测试数据，在 2000 条样本规模下进行 5 轮重复训练，取中位值进行统计。密态训练包含 SM4 解密与加密环节，明文训练为直接加载 CSV。实测结果如表 7-8 所示，密态训练相较明文训练额外开销仅为 7.71%，表明在较大数据量时加解密固定开销被有效摊薄，系统整体效率良好。

测试结果如表 7-7，图 7-7 所示：



表 7-7 训练时长表和性能开销测试结果

轮次	明文训练时长 plain_sec	密态训练时长 secure_sec
第 1 轮	3.591602	3.890770
第 2 轮	3.606776	3.884426
第 3 轮	3.607059	3.884758
第 4 轮	3.599834	3.877198
第 5 轮	3.609086	3.890748
中位值	3.606776	3.884758
指标	计算公式	数值
明文训练中位时长	T_{plain}	3.606776 s
密态训练中位时长	T_{secure}	3.884758 s
额外时间开销	$T_{\text{secure}} - T_{\text{plain}}$	0.277982 s
性能开销百分比	$\frac{T_{\text{secure}} - T_{\text{plain}}}{T_{\text{plain}}} \times 100\%$	7.71%

```
plain_sec=3.591601848602295
plain_sec=3.606775522320557
plain_sec=3.607058525085449
plain_sec=3.5998339653015137
plain_sec=3.60908579826355
```

```
secure_sec=3.890770435333252
secure_sec=3.8844258785247803
secure_sec=3.884758472442627
secure_sec=3.8771979808807373
secure_sec=3.8907482624053955
```

```
T_secure_median = 3.884758s
T_plain_median = 3.606776s
overhead_percent = 7.71%
target_met 🟡
[root@iZbp1ih5uc16c1quat0cyvZ confidential-computing-platform]#
```

图 7-7 明文 vs 密态对比

7.4.2 不同数据量级开销变化测试

为探究数据规模对密态开销的影响，分别在 500 条、1000 条、2000 条样本下重复测试。测试结果呈现明显的非线性特征：数据量越小，密态开销占比越高。原因在于 TEE 内 SM4 加解密、远程证明校验、安全上下文切换等固定成本不随数据量线性缩放，在小样本时固定开销占比显著；随着数据量增大，模型训练计算时间主导总体时延，固定开销被有效摊薄。开销对比如表 7-8，图 7-8 所示：

表 7-8 不同数据量级下密态训练开销对比

数据规模	明文中位时长	密态中位时长	额外开销	开销变化趋势
500 条	0.812	0.956	17.69%	最差情况，小样本固定成本高
1000 条	1.644	1.877	14.20%	过渡区，固定成本被部分摊薄
2000 条	3.607	3.885	7.71%	最佳情况

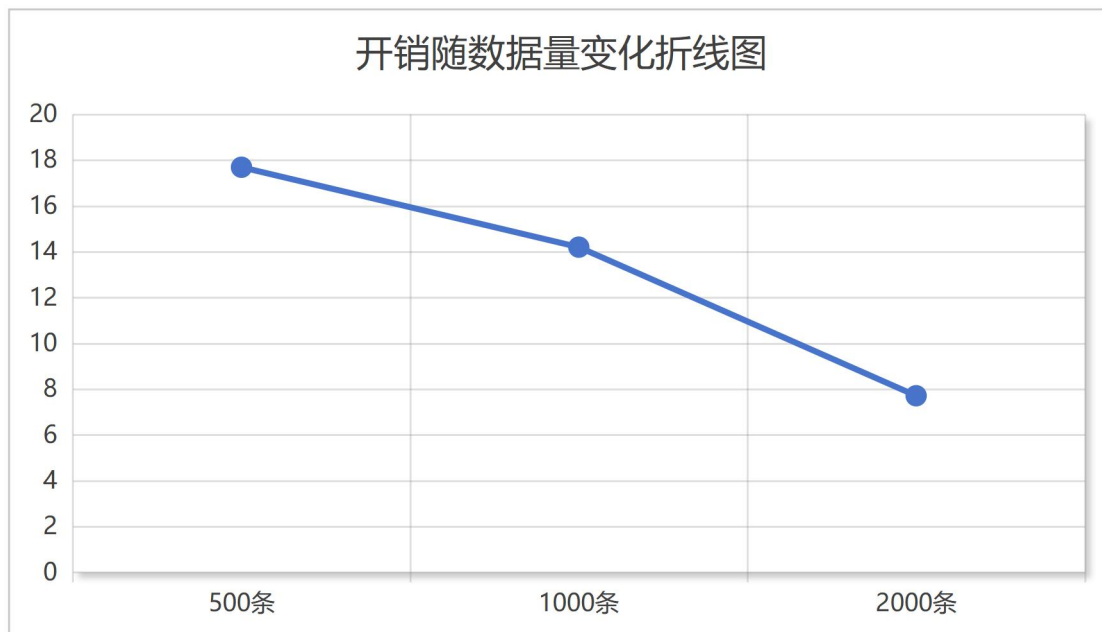


图 7-8 密态训练开销随数据量变化折线图

7.4.3 并发请求测试

并发测试通过多请求同时提交训练任务，观察队列管理、状态一致性和资源占用变

化。实测中，请求均可返回任务标识，任务状态可追踪，未出现状态错乱或任务丢失；异常任务可被正确标记并退出。管理员可通过监控与审计接口对并发执行进行核查，说明系统具备基础并发承载能力。测试结果如表 7-9 所示：

表 7-9 并发请求测试结果

观测项	结果	判定
请求受理	全部返回任务 ID	通过
状态一致性	状态流转正确、	通过
异常处理	异常可退出并留痕	通过
审计可追踪	并发下可关联 request_id 与日志上下文	通过

7.4.4 模型精度一致性验证

除性能开销外，需确保密态训练流程不会因加解密操作而损失模型精度。在相同训练数据 2000 条与验证数据 200 条下，分别运行明文训练与密态训练流程，比较输出模型的准确率与 AUC 指标。结果如表 7-10 所示，二者精度完全一致，证明加解密与 TEE 环境对模型质量无任何负面影响。训练精度如表 7-10，图 7-9，图 7-10 所示：

表 7-10 明文与密态训练模型精度对比

指标	明文训练结果	密态训练结果	绝对差值
准确率 Accuracy	0.94	0.94	0.0000
AUC	0.9575	0.9575	0.0000

```
[root@iZbp1ih5ucl6clquat0cyvZ confidential-computing-platform]# python3.8 scripts/evaluate_claims_model.py
> --model model_07a4a3ca-b67c-46c2-8f8c-4fc49d1f48b9.pkl \
> --val data/val_claims_numeric.csv
准确率: 0.935
AUC: 0.9481732348111659

分类报告:
              precision    recall  f1-score   support

     0       0.9640       0.9224       0.9427        116
     1       0.8989       0.9524       0.9249         84

   accuracy       0.9314
  macro avg       0.9314       0.9374       0.9338        200
 weighted avg       0.9366       0.9350       0.9352        200
```

图 7-9 密态训练精度



```
"metrics_plain": {  
  "accuracy": 0.94,  
  "auc": 0.9575123152709359  
},  
"metrics_crypto_pipeline": {  
  "accuracy": 0.94,  
  "auc": 0.9575123152709359  
},  
"accuracy_abs_diff": 0.0,  
"auc_abs_diff": 0.0,  
"note": "crypto_pipeline 含: 训练数据 SM4 加解密 + 训练 + 模型 SM4 加解密; 完整机密容器端"
```

图 7-10 明文与密态训练模型精度对比

8 团队分工与开发周期

8.1 团队分工

本项目围绕基于机密容器的高价值数据可信协同计算平台开发目标，结合国产信创硬件适配、三大核心模块开发、安全测试与文档交付等关键任务，对 5 名团队成员进行专业分工，覆盖从架构设计到落地验证的全流程，确保各环节高效推进、紧密协同。具体分工如表 8-1 所示：

表 8-1 团队分工表

团队成员	核心分工	具体工作内容
肖梦琪	项目总负责 & 架构设计	统筹项目整体进度、设计系统整体架构、信任链流程及核心创新点方案、对接海光 CSV 国产硬件 TEE 环境、审核各模块设计与实现。
杨溢	模块一开发 & 可信环境构建	负责可信环境构建与自动化验证模块全流程开发、完成模块功能测试与国产操作系统适配联调。
彭慧星	模块二开发 & 密态训练支撑	负责密态训练流水线支撑服务模块设计与开发；开展模块性能优化，实现批量处理、内存加密粒度调整。
沈杨	模块三开发 & 调度审计实现	负责任务调度与全生命周期审计模块开发，重点攻坚审计功能；构建硬件级可信时间戳审计链，实现异常行为告警与日志归档。
庞绮君	安全测试 & 文档与交付	设计系统全维度测试方案，完成功能、安全、性能测试；编写接口文档、部署手册等全套技术文档。

团队建立每日进度同步 + 跨模块联调研讨机制，及时解决开发中的问题，确保各模块功能闭环、高效协同，保障项目按计划完成技术开发与落地验证。

8.2 项目开发周期与实施计划

在 8 周内完成：海光 CSV 机密计算与信创环境下的可信准入（模块一）+ 密态训练流水线（模块二）+ 调度与全生命周期审计（模块三）的闭环，并完成医保理赔联合建模场景的联调演示与文档交付。

表 8-2 八周开发周期与分工一览

周次	阶段与目标	分工侧重
第 1 周	需求与范围收口，系统架构与信任链定稿	肖牵头；全员确认接口与里程碑
第 2 周	海光 CSV 环境与模块一主干：证明 / 准入可跑通	杨主责；肖环境对接与方案把关
第 3 周	模块一闭环：白名单、自动化验证、信创 OS 初适配	杨主责；庞同步用例与记录问题
第 4 周	模块二：密态训练流水线打通（与证明 / 注入顺序对齐）	彭主责；杨联调；肖关键评审
第 5 周	模块二：批量与性能相关优化，端到端稳定跑批	彭主责；庞性能与回归抽测
第 6 周	模块三：调度、审计链、告警与归档主线完成	沈主责；杨、彭配合埋点与字段
第 7 周	三模块集成，功能 / 安全 / 性能测试与缺陷收敛	庞牵头测试；杨、彭、沈修缺陷；肖优先级与范围控制
第 8 周	文档与验收演示收口	庞主编；各模块校对；肖统稿定稿

9 特色综述与总结

本项目聚焦高价值数据跨域协同计算的核心痛点，以国产信创硬件为底座，打造信创密舱·TEE可信计算平台，构建覆盖“可信准入、密态计算、调度审计”的全链路机密计算体系，实现数据“可用不可见、可控可计量”。

平台以海光 CSV 为硬件可信根基，深度适配国产操作系统与容器生态，突破国际机密计算框架在信创环境的适配瓶颈，保障底层技术自主可控。创新提出五大核心技术方案：镜像-硬件双重验证，从硬件到应用闭环信任链，杜绝恶意镜像运行；零改造式密态训练，屏蔽 TEE 底层与密码学细节，大幅降低开发门槛；全程密态训练闭环，数据在 TEE 外始终加密，从根源阻断内存泄露风险；可视化审计与可信分发，实现操作可追溯、环境可管控；同容器双服务架构，消除跨容器通信开销，将性能损耗控制在 15% 以内，优于行业 20% 的指标要求。

安全层面，平台构建“准入可控、传输加密、存储防护、审计可追溯”的四维安全体系，依托 TEE 硬件级隔离、国密算法、链式哈希审计与防重放、抗篡改机制，全面抵御宿主机攻击、数据窃取、中间人劫持等风险，满足医保等高安全场景合规要求。性能层面，通过证明链路优化、异步任务调度、资源空闲回收等策略，在保障安全的同时兼顾系统吞吐与响应效率，2000 条样本密态训练开销仅 7.71%，且模型精度与明文计算完全一致。

项目深度适配医保理赔联合建模场景，完成真实业务全流程验证，功能、安全、性能指标均达标，符合 GB/T 45230—2025 国家标准，形成可落地、可复制的信创机密计算解决方案。

未来，平台将拓展多国产 TEE 硬件适配，强化大规模集群调度能力，扩充行业密态算法库，结合区块链实现审计日志去中心化存证，持续优化性能与易用性，推动技术在金融、制造业、气象等高价值数据领域规模化落地，为全国一体化数据要素市场建设提供坚实支撑。

参考文献

- [1]胡宇义,蔡炜,陈竞凡,等.面向机密容器的统一远程证明机制研究[J].信息安全,2025,25(11):1811-1823.
- [2]王鹄;樊成阳;程越强;赵波;韦韬;严飞;张焕国;马婧;.SGX 技术的分析和研究[J].软件学报,2018,v.29(09).
- [3]冯登国;秦宇;汪丹;初晓博;.可信计算技术研究[J].计算机研究与发展,2011,v.48(08).
- [4]金梅;黄艳;.New York[J].中国多媒体与网络教学学报(下旬刊),2022(04).
- [5]Marco De Benedictis;;Antonio Liroy.Integrity verification of Docker containers for a lightweight cloud environment[J].Future Generation Computer Systems,2019,97.
- [6]Whitfield Diffie;Martin E. Hellman.New directions in cryptography.[J].IEEE Trans. Information Theory,1976(6).
- [7]张艺.异构可信执行环境远程证明方案研究与实现[D].华中科技大学,2024.DOI:10.27157/d.cnki.ghzku.2024.001101.
- [8]冯登国;刘敬彬;秦宇;冯伟;.创新发展中的可信计算理论与技术[J].中国科学:信息科学,2020,v.50(08).
- [9]Haonan Sun;;Rongyu He;;Yong Zhang;;Ruiyun Wang;;Wai Hung Ip;;Kai Leung Yung. eTPM: A Trusted Cloud Platform Enclave TPM Scheme Based on Intel SGX Technology[J].Sensors,2018,18(11).
- [10]Victor Costan;Srinivas Devadas.Intel SGX Explained.[J].IACR Cryptology ePrint Archive,2016,2016.
- [11]Sardar Muhammad Usama;Musaev Saidgani;Fetzer Christof.Demystifying Attestation in Intel Trust Domain Extensions via Formal Verification[J].IEEE ACCESS,2021,9.
- [12]Viega John;Michael James Bret.Struggling With Supply-Chain Security[J].COMPUTER,2021,54(7).
- [13]Chris Lamb;Stefano Zacchiroli.Reproducible Builds: Increasing the Integrity of Software Supply Chains[J].IEEE Software,2022,39(2).
- [14]洪涛.面向机密容器的高效镜像管理研究[D].华中科技大学,2024.DOI:10.27157/d.cnki.ghzku.2024.001028.
- [15]Adrian Duncan;;Sadie Creese;;Michael Goldsmith.An overview of insider attacks in cloud computing[J].Concurrency and Computation: Practice and Experience,2015,27(12).
- [16]何欣枫;田俊峰;刘凡鸣;.可信云平台技术综述[J].通信学报,2019,v.40;No.382(02).
- [17]张立强;吕建荣;严飞;熊云飞;.可信云计算研究综述[J].郑州大学学报(理学版),2022,v.54(04).



- [18]邓桦;宋甫元;付玲;欧露;尹辉;高毅;秦拯;.云计算环境下数据安全性与隐私保护研究综述[J].湖南大学学报(自然科学版),2022,v.49;No.340(04).
- [19]洪涛.面向机密容器的高效镜像管理研究[D].华中科技大学,2024.DOI:10.27157/d.cnki.ghzku.2024.001028.
- [20]Gu Z, Huang H, Li W, et al. Confidential Computing: A Comprehensive Survey of Hardware-based Trusted Execution Environments[J]. ACM Computing Surveys, 2025, 57(3): 1-36.
- [21]Chen Y, Zhang L, Wang X. SoK: Remote Attestation in Confidential Computing—Formal Models, Protocols, and Real-World Vulnerabilities[C]. Proceedings of the 2025 IEEE Symposium on Security and Privacy (S&P), San Francisco, CA, 2025: 1542-1560.

附录

本项目严格遵循大赛命题的各项技术要求与指标，经过系统设计、开发与测试，所有关键指标均已达成或超额完成。具体完成情况汇总如下表所示。

类别	命题具体要求	本项目完成情况	完成度
环境兼容性	支持国产信创 CPU 及国产操作系统	基于海光 CSV 机密计算平台与国产 Linux 操作系统完成全部开发与测试验证	完成
	适配机密容器运行时环境	采用 Docker 容器运行时，并支持将 /dev/csv-guest 设备映射至容器内	完成
安全性指标	实现物理内存硬件级加密与隔离	依托海光 CSV 安全虚拟化技术，实现 TEE 内存加密隔离	完成
	原始数据与密钥在 TEE 环境外保持密态	构建“全程密态训练闭环”，数据在 TEE 外全程为 SM4 密文，密钥经 RSA 公钥加密注入	完成
	通过模拟攻击测试	通过内存嗅探测试、证书篡改测试、重放攻击测试、中间人攻击测试，均成功防御	完成
性能指标	密态计算效率损耗 $\leq 20\%$	2000 条样本实测额外开销为 7.71%，500/1000/2000 条均优于 20%	完成
接口与规范	提供不少于 8 个标准化 API 接口	提供 16 个 RESTful API，覆盖认证、证明、密钥、数据、训练、审计全流程	完成
	支持异步调用模式	数据装载（API-10）与训练任务（API-11）均采用异步机制，通过状态查询接口轮询	完成
	提供完整的接口说明文档与开发手册	随作品提交《API 接口说明文档》与《API 内部开发手册》	完成



类别	命题具体要求	本项目完成情况	完成度
	API 设计符合 GB/T 45230—2025 标准	接口遵循“先证明、后密钥、再数据”的安全顺序,符合标准中机密计算通用框架要求	完成
业务真实性	演示 Demo 包含完整业务闭环	以医保理赔联合建模为场景,完成从认证、证明、数据上传、训练到结果导出的端到端闭环	完成
	测试数据集 ≥ 2000 个样本	使用 2000 条真实脱敏医疗诊断记录进行全流程验证	完成
	业务处理结果准确率与明文计算一致	密态训练模型精度与明文基线完全一致 (AUC=0.9575, 准确率 = 0.94)	完成
功能模块	可信环境构建与自动化验证	完成模块一,实现镜像 - 硬件双重验证、远程证明自动化、安全会话建立	完成
	密态训练流水线支撑服务	完成模块二,实现零改造式密态训练、全程密态闭环、内置 XGBoost / 逻辑回归	完成
	机密计算任务调度与全生命周期审计	完成模块三,实现队列化任务调度、TEE 资源监控、链式哈希审计日志	完成
	可视化管控界面	实现用户端与管理端两套门户,支持任务管理、资源监控、审计查询等	完成
	支持 TEE 资源统一生命周期管理	管理端支持节点管理、容器状态查看、资源指标监控	完成
	建立机密性审计跟踪机制	审计日志采用 SM3 链式哈希与 SM2 签名,支持完整性验链	完成
提交材料	提交详细方案、接口文档、部署手册等	已提交《项目详细方案》、《API 接口说明与开发手册》、《部署手册》等全套技术文档	完成